

从 0 开始，
用命令行打开更大的世界。

*More
Than Linux.*

Hola Euler >_

openEuler Linux 从入门到实战

Alex 著

写给每一个想真正学会 Linux 的你

*敲下的每一行命令，
都是在靠近一个更好的自己。*

Hola Euler
openEuler Linux 从入门到实战

Alex
著

>_



扉页

Hola Euler

openEuler Linux：从命令行到服务器实战

Alex 著

基于 openEuler 24.03 LTS SP4 • x86_64 • Bash

v0.2.2 • 实机修订审阅版 • 2026 年 9 月

从一个目录开始，到一次能够解释的服务部署。

版权与协作说明

本稿供作者 Alex 与获授权读者审阅，已按作者要求保存到私有 GitHub 仓库。书名、封面和章节安排仍可由作者修改。开放许可尚未决定，本项目未放置 LICENSE，也未把任何第三方许可证自动套用于全书。

本书在资料整理、初稿撰写、内容修改、练习设计、技术核查及排版过程中使用人工智能工具辅助。作者 Alex 对本书的结构、内容选择、技术验证及最终发布版本负责。

Human-AI Collaboration 是本书公开说明的一部分，AI 不作为署名作者。

本书为独立创作的学习材料，与 openEuler 官方教材、官方出版物或华为官方出版物无关，不表示任何组织背书。Linux、openEuler、Java 等名称用于识别相关技术，其商标归属相应权利人。

正文示意图由项目原创绘制。立体书封由作者提供并指定用于仓库首页，本版电子书也采用该封面。封面副标题“从入门到实战”与扉页完整题名指向同一本书。字体使用本机可用字体进行内部排版，不随源稿分发字体文件，公开出版前复核字体授权。

本稿在 openEuler 24.03 LTS SP4 上完成了普通用户文件与权限、脚本、Java 构建、systemd、SSH 隧道和升级回滚测试。Spring Boot 构建使用独立镜像配置。重启恢复按用户要求未执行，安装器重装流程与带密码的 sudo 交互仍未验收。具体条件、早期失败和复测结果保存在运行报告中，作者审核尚待完成。

对读者的话

这本书由 Alex 发起，希望帮助第一次系统学习 Linux 的读者解决一个具体问题。跟着示例输入时觉得懂了，关掉教材以后，面对一个新目录却不知道敲哪条命令。

因此，每章从一件小事出发。先在自己的实验目录操作，看看发生了什么，再解释名称、参数和原理。练习会要求你合上前文，从需求写命令；后面的项目也会反复用到前面学过的路径、权限和日志。

编写时查阅了 openEuler、GNU、Bash、systemd 和相关软件的官方资料。资料之间有版本差异，官方文档中也可能留下旧版本指引。本书固定实验版本，把尚未实测的事项留下明确标记，让技术核查有据可查。

本书使用 AI 参与协作。资料整理和初稿生成之后，仍需要逐条检查技术解释、命令条件和教学次序。Alex 审核并决定最终内容，负责正式发布版本。本轮已从头审查章节、练习与示例，修订了执行前提和错误处理。命令审查记录与目标系统实测分开保存，尚未完成的实测继续明确标注。

学习时请保留一次失败。写清当时在哪个目录、用哪个用户、运行什么命令，再试着找到一条能够验证猜想的查询。第 24 章完成以后，你会拥有一份自己的部署记录，它应当足够清楚，让过几天的你还能据此重做。

目录

三级目录 • 可点击跳转 • 页码为 PDF 页序

扉页	2
版权与协作说明	3
对读者的话	4
第 0 章 如何使用本书	19
0.1 从一件小事开始	19
0.1.1 今天要留下什么	19
0.1.2 本书实验约定	19
0.2 怎样读一条命令	20
0.2.1 空格把任务分成几部分	20
0.2.2 看结果而非只等一句成功	20
0.3 给记忆留一次工作机会	21
0.3.1 四次短复习	21
0.3.2 第一次学习记录	21
0.4 本章练习	22
第 1 章 Linux 与 openEuler	23
1.1 让一台虚拟机能够工作	23
1.1.1 内核与发行版各做什么	23
1.1.2 确认要下载的版本	24
1.2 完成安装与第一次登录	24
1.2.1 新建专用虚拟机	24
1.2.2 用户和管理员	24

1.3 证明自己进入了哪台系统	25
1.3.1 三条只读命令	25
1.3.2 保存一个干净起点	25
1.4 本章练习	26
第 2 章 Terminal、Shell 与 Bash	27
2.1 光标后面发生了什么	27
2.1.1 一个窗口，三层工作	27
2.1.2 先试一个容易看清结果的命令	27
2.2 分清命令来自哪里	28
2.2.1 内建命令与外部程序	28
2.2.2 请求帮助	28
2.3 少敲一点，少错一点	29
2.3.1 补全和历史	29
2.3.2 有意拼错一次	29
2.4 本章练习	29
第 3 章 Linux 文件系统与路径	30
3.1 我现在在哪里	30
3.1.1 一棵从斜杠开始的目录树	30
3.1.2 工作目录是相对路径的起点	30
3.2 同一个文件可以有几种写法	31
3.2.1 绝对路径与相对路径	31
3.2.2 走错路时怎样检查	32
3.3 大小写与空格都属于名字	32
3.3.1 让 Shell 把名字看完整	32
3.3.2 返回家目录再重来	32
3.4 本章练习	33
第 4 章 文件与目录操作	34

4.1 为一份文件建立副本	34
4.1.1 创建并看见它	34
4.1.2 复制时源在前、目标在后	34
4.2 改名字与搬位置	35
4.2.1 mv 的两种常见使用方式	35
4.2.2 复制目录需要明确递归	35
4.3 删除之前确认对象	36
4.3.1 从一个文件开始	36
4.3.2 两种链接	36
4.4 本章练习	37
第 5 章 查看与处理文本	38
5.1 文件有内容以后怎样阅读	38
5.1.1 准备一份小日志	38
5.1.2 less 让阅读可以停下来	38
5.2 只看需要的几行	39
5.2.1 头部和尾部	39
5.2.2 跟随追加内容	39
5.3 把重复内容整理出来	39
5.3.1 顺序决定去重结果	39
5.3.2 切出简单字段	40
5.4 本章练习	40
第 6 章 Vim	41
6.1 能进去，也能出来	41
6.1.1 从一份自己的笔记开始	41
6.1.2 如果只是误打开	42
6.2 让修改动作可控	42
6.2.1 移动、删除与撤销	42
6.2.2 查找比逐行移动更可靠	42

6.3 报错往往与文件有关	42
6.3.1 保存失败先检查路径与权限	42
6.3.2 交换文件提示	43
6.4 本章练习	43
第 7 章 用户与用户组	44
7.1 程序究竟以谁的身份运行	44
7.1.1 看看当前账户	44
7.1.2 查询账户数据库	44
7.2 临时管理权限与切换身份	45
7.2.1 sudo 的范围	45
7.2.2 没有授权时怎样继续	45
7.3 创建一个可辨认的练习账户	45
7.3.1 先检查名称，再创建	45
7.3.2 附加组不能误覆盖	46
7.4 本章练习	47
第 8 章 Linux 权限	48
8.1 为什么自己创建的文件也可能不能写	48
8.1.1 看懂三组 rwx	48
8.1.2 数字来自三个位的组合	49
8.2 目录权限与文件权限不同	49
8.2.1 能列名字不等于能读取内容	49
8.2.2 使用符号方式增加一项	49
8.3 所有者和默认权限	49
8.3.1 chown 改变的是归属	49
8.3.2 umask 从新建请求中去掉权限	50
8.4 权限报错的检查顺序	50
8.5 本章练习	51

第 9 章 DNF 与 RPM	52
9.1 一个命令不存在时该做什么	52
9.1.1 程序、包与仓库	52
9.1.2 先看系统提供了什么	52
9.2 完成一次可审查的安装	52
9.2.1 查看候选包	52
9.2.2 安装后的证据	53
9.2.3 手册程序与文档包分开检查	53
9.3 更新、移除与来源	54
9.3.1 变更前保留可恢复状态	54
9.3.2 不混用不匹配的软件源	54
9.4 本章练习	54
第 10 章 进程与任务	55
10.1 文件中的程序怎样变成正在做事的进程	55
10.1.1 观察一个会等待的程序	55
10.1.2 工作目录、环境与打开的文件	56
10.2 前台、后台与作业	56
10.2.1 当前 Shell 管理的作业	56
10.2.2 让进程结束	56
10.3 观察资源使用	57
10.3.1 从动态列表找线索	57
10.3.2 进程存在与服务可用	57
10.4 本章练习	57
第 11 章 systemd 与服务	58
11.1 关掉终端以后谁照看程序	58
11.1.1 服务与管理器	58
11.1.2 确认管理器	58
11.2 查询与变更要分开理解	59

11.2.1 start 与 enable 不回答同一个问题	59
11.2.2 读取单元和日志	59
11.3 最小单元文件怎样表达一个程序	60
11.3.1 读懂后续部署模板	60
11.3.2 修改单元与修改程序	60
11.4 本章练习	61
第 12 章 Linux 网络基础	62
12.1 为什么程序说连不上	62
12.1.1 先把四个对象分开	62
12.1.2 逐层观察	62
12.2 从本机到应用逐步检查	63
12.2.1 地址、路由与名称	63
12.2.2 检查监听端口与 HTTP	63
12.3 NetworkManager 的配置与状态	64
12.3.1 一个配置可以暂时不被启用	64
12.3.2 配置变更应保留退路	64
12.4 本章练习	64
第 13 章 SSH 与远程服务器	65
13.1 在自己的电脑操作虚拟机	65
13.1.1 客户端与服务端	65
13.1.2 第一次连接为什么问指纹	66
13.2 用公钥登录	66
13.2.1 私钥留在客户端	66
13.2.2 安装公钥后另开窗口验证	66
13.3 传文件与建立隧道	67
13.3.1 scp 的端口是大写 P	67
13.3.2 只让应用监听服务器本机	67
13.3.3 登录成功，转发仍可能被拒绝	68

13.4 本章练习	69
第 14 章 压缩、打包与归档	70
14.1 把多个文件装进一个文件	70
14.1.1 归档与压缩分开看	70
14.2 恢复要在新目录中演练	70
14.2.1 看过清单再解开	70
14.2.2 文件存在不等于备份可靠	71
14.3 校验内容变化	71
14.3.1 为归档生成摘要	71
14.3.2 命令写反会改变任务	71
14.4 本章练习	72
第 15 章 磁盘、分区与文件系统	73
15.1 路径背后是怎样的存储	73
15.1.1 设备、分区和文件系统	73
15.1.2 只读查看已经存在的布局	73
15.2 明明还有空间为什么写不进去	74
15.2.1 块空间与 inode 都可能用完	74
15.2.2 找到问题再回收空间	74
15.3 挂载点会暂时遮住原内容	74
15.3.1 一次安全的内存文件系统实验	74
15.3.2 不在初学实验里格式化真实盘	75
15.4 本章练习	75
第 16 章 Bash 环境与环境变量	76
16.1 同样的命令为什么在另一个终端不一样	76
16.1.1 变量先属于当前 Shell	76
16.1.2 export 把值传给后来的子进程	76
16.2 PATH 怎样找到程序	76

16.2.1 按目录顺序查找	76
16.2.2 临时设置比全局修改更容易检查	77
16.3 启动文件何时读取	77
16.3.1 交互与登录是两个维度	77
16.3.2 source 会在当前 Shell 中执行	78
16.4 本章练习	78
第 17 章 管道与重定向	79
17.1 屏幕上的两种文字	79
17.1.1 为输入输出编号	79
17.1.2 把正常结果保存到文件	79
17.2 把错误单独保留	80
17.2.1 一个命令同时有成功和失败对象	80
17.2.2 重定向从左到右处理	80
17.3 让一个程序的输出成为另一个的输入	81
17.3.1 管道连接的是流	81
17.3.2 保存一份同时继续传递	81
17.4 多行输入与退出状态	82
17.4.1 here-document 让模板易读	82
17.4.2 管道最后一段成功不代表前面都成功	82
17.5 本章练习	82
第 18 章 grep、find 与文本搜索	83
18.1 内容和名字是两种搜索	83
18.1.1 在日志里找一段文字	83
18.1.2 正则表达式从三个符号开始	83
18.2 按文件条件查找	84
18.2.1 find 沿目录树访问	84
18.2.2 搜索范围本身就是保护	84
18.3 把路径安全交给下一个命令	84

18.3.1 不按空格拆文件名	84
18.3.2 换行也可能属于文件名	85
18.4 本章练习	85
第 19 章 Shell Script 入门	86
19.1 把重复步骤写进一个文件	86
19.1.1 第一份脚本	86
19.2 成功与失败需要明确判断	87
19.2.1 退出状态	87
19.2.2 参数校验写在操作之前	87
19.3 循环与函数	88
19.3.1 保留每个参数的完整边界	88
19.3.2 函数只组合清楚的工作	88
19.4 调试与执行边界	88
19.5 本章练习	89
第 20 章 Shell Script 实战	90
20.1 为自己的笔记生成可恢复归档	90
20.1.1 先定义输入、输出与失败条件	90
20.1.2 把脚本保存为 backup-notes.sh	91
20.2 检验脚本是否真的可靠	92
20.2.1 三类测试	92
20.2.2 这份脚本的边界	92
20.3 为应用写一个小型健康检查	92
20.3.1 只检查明确的接口	92
20.3.2 故障时只提供证据	93
20.4 本章练习	93
第 21 章 Linux 开发环境	94
21.1 让代码、产物和记录各有位置	94

21.1.1 从一个项目目录开始	94
21.1.2 构建工具与运行时	94
21.2 用 Git 记录有意义的修改	94
21.2.1 本地仓库不等于公开发布	94
21.2.2 配置自己的署名	95
21.3 可复现比依赖手工记忆更可靠	95
21.3.1 记录环境	95
21.3.2 把构建和部署分开	95
21.4 本章练习	96
第 22 章 Java 与 Linux	97
22.1 先准备完整的 JDK	97
22.1.1 确认运行与编译能力	97
22.2 用标准 JDK 建立第一个 HTTP 服务	98
22.2.1 完整程序 LabServer.java	98
22.2.2 编译、打包、前台运行	100
22.3 把同一目标换成 Spring Boot	100
22.3.1 固定一份独立项目	100
22.3.2 一个控制器与两个接口	101
22.3.3 构建前核对 Maven	102
22.3.4 仓库拒绝下载时保留证据	103
22.4 本章练习	104
第 23 章 Linux 服务器部署基础	105
23.1 从“运行了”走到“可以维护”	105
23.1.1 部署要留下哪些东西	105
23.1.2 代码与数据需要不同所有者	105
23.2 启动前确认四项条件	106
23.2.1 程序、路径、身份和端口	106
23.2.2 防火墙与安全策略各做什么	106

23.3 先限制在本机，再决定公开入口	106
23.3.1 SSH 隧道足够完成本书实验	106
23.3.2 端口开放也需要证明应用正确	107
23.4 日志、重启和升级	107
23.4.1 让日志能解释一次失败	107
23.4.2 回滚恢复的是哪个部分	107
23.5 本章练习	107
第 24 章 综合项目	108
24.1 从一台新安装的 openEuler 开始	108
24.1.1 项目目标与验收证据	108
24.1.2 先检查本章名称是否空闲	108
24.2 创建身份与发布目录	108
24.2.1 使用无交互登录需求的服务账户	108
24.2.2 上传与安装产物	109
24.3 先用服务身份前台运行	109
24.3.1 把程序错误与管理器错误分开	109
24.3.2 写入专用单元文件	110
24.3.3 等待应用真正就绪	111
24.4 加载、启动与验收	112
24.4.1 先验证单元语法	112
24.4.2 五份证据共同构成验收	112
24.5 更新到 v2，再回到 v1	113
24.5.1 构建一个能辨认的新版本	113
24.5.2 回滚需要再次验证	114
24.6 遇到失败时按证据收窄范围	114
24.6.1 常见故障与第一项检查	114
24.6.2 保留实验结果	115
24.7 本章练习	115

练习答案	116
第 0 章	116
第 1 章	116
第 2 章	116
第 3 章	117
第 4 章	117
第 5 章	117
第 6 章	118
第 7 章	118
第 8 章	119
第 9 章	119
第 10 章	119
第 11 章	120
第 12 章	120
第 13 章	120
第 14 章	121
第 15 章	121
第 16 章	121
第 17 章	122
第 18 章	122
第 19 章	122
第 20 章	123
第 21 章	123
第 22 章	123
第 23 章	124
第 24 章	124
附录 A 核心命令速查	125

附录 B 命令名称与英文关联	127
附录 C 常见报错速查	129
附录 D openEuler、Ubuntu 与 CentOS 常见差异	131
附录 E Linux 后续学习路线	132
E.1 把本书项目再做两次	132
E.2 选择一个真实方向	132
E.3 读一份手册并提出一个问题	132
参考资料与版本核验	133
S01 openEuler 官方下载与 SP4 镜像目录	133
S02 openEuler SP4 快速入门与系统信息	133
S03 GNU Bash Reference Manual	133
S03a Bash Pipelines	133
S03b Bash Redirections	134
S04 GNU Coreutils Manual	134
S04a GNU touch invocation	134
S04b GNU cp invocation	134
S04c GNU chmod invocation	134
S05 Vim User Manual	134
S06 openEuler 常用技能与账户工具手册	135
S07 DNF Command Reference	135
S08 RPM 官方查询手册	135
S09 systemd 官方服务与执行环境手册源码	135
S10 systemctl 与 journalctl 官方手册源码	135
S11 openEuler 网络配置与 NetworkManager 手册	136
S12 Linux man-pages 与内核凭据说明	136

S13 curl 官方命令文档 136

S14 OpenSSH 官方手册 136

S15 GNU tar Manual 137

S16 util-linux 工具手册 137

S17 GNU Grep Manual 137

S18 GNU Findutils Manual 137

S19 Git 官方文档 137

S20 JDK 21 工具与 HTTP API 138

S21 Spring Boot 3.5 系统要求 138

S22 Spring Boot 首个应用教程 138

S23 firewalld 官方 firewall-cmd 手册 138

S24 SELinuxProject getenforce 手册源码 138

S25 Ubuntu 与 CentOS 官方项目资料 139

检索限制与交叉检查 139

S26 Hello 算法的阅读与练习组织 139

S27 openEuler 中文运维文档交叉核对 139

S28 GNU 目标目录语义 140

S29 SSH 转发失败与 HTTP 响应 140

S30 Maven 工具链选择与独立镜像配置 140

S31 SSH 服务端转发策略 140

第 0 章 如何使用本书

本章任务 | 建立自己的实验目录，退出后还能重新找到它。

0.1 从一件小事开始

0.1.1 今天要留下什么

合上教材之后，能否自己建立一个目录，把文件放进去，再找到它？本书从这样的小任务开始。每章都要留下可以检查的结果，例如一个文件、一份日志或一个能够访问的服务。看懂一页之后，请把视线从书上移开，重新做一次。

本书按文件操作、系统管理、脚本与部署展开。前半部分不要求懂网络协议和操作系统原理。遇到第一次出现的概念，先用眼前的实验理解它。后面的章节会继续使用前面的目录、权限和文本处理知识。

0.1.2 本书实验约定

目标环境为 openEuler 24.03 LTS SP4，处理器架构为 x86_64，交互式命令解释器为 Bash。请在虚拟机中学习。虚拟机是一台由软件提供的计算机，它有自己的系统和虚拟磁盘；关掉虚拟机不会自动撤销磁盘上的修改，快照也不能代替长期备份。

全书把普通练习放在当前用户家目录下的 linux-lab 中。家目录用于保存自己的文件。第 1 章完成系统准备，第 3 章会解释目录路径。尚未准备环境的读者，可以先阅读，再回来执行下面三行。

```
BASH
mkdir -p ~/linux-lab
cd ~/linux-lab
pwd
```

mkdir 关联 make directory，创建目录。-p 允许所需父目录一并创建，目录已存在时不因此报错。cd 关联 change directory，改变当前位置。pwd 关联 print working directory，打印当前工作目录。~ 在这里由 Bash 展开为当前用户的家目录。三行分别输入，每行按一次回车。最后应看到一个以 /linux-lab 结尾的路径，用户名和前面的目录由你的系统决定。

实验边界 | 普通文件操作只针对自己建立的实验文件。任何出现 `sudo` 的步骤都涉及更高权限，应先读解释再操作。正文代码没有提示符，方便复制；终端中已经显示的 `$` 或 `#` 不属于要输入的命令。

0.2 怎样读一条命令

0.2.1 空格把任务分成几部分

TEXT

```
cp hello.txt backup.txt
| |
| | +-- 目标名称
| +----- 源文件
+----- cp, 复制
```

`cp` 关联 `copy`。整行要求读取 `hello.txt`，并写出 `backup.txt`。暂时不用运行，这两个文件将在第 4 章准备。命令的第一个词通常指定做事的程序，后面是选项与操作对象。选项改变做事方式，操作对象说明处理谁。它们都是命令行参数的一部分。

TEXT

```
cp [选项] <源> <目标>
```

这种写法是说明格式。方括号表示该部分可选，尖括号表示需要替换的内容，都不要照敲。真实 Shell 中这些符号还有别的含义，第 17 章会解释。某些命令支持 `-r` 这样的短选项与 `--recursive` 这样的长选项；是否等价，要看该命令自己的手册。一个命令的 `-r` 不能推导出另一个命令的 `-r`。

提示 | 先读代码块前面的执行位置和准备条件，再逐行输入。故意拼错、访问不存在文件等实验预计会失败；不要把整章代码一次粘贴成脚本。

0.2.2 看结果而非只等一句成功

许多 Linux 命令成功时没有文字输出。创建目录后，提示符重新出现，只能说明前一个命令已经结束；应再查看目录确认结果。反过来，文字很多也不代表成功。出错时保留完整命令、当前路径、报错和当时的用户身份，这些信息比“不能用”更有帮助。

本书不把推演结果伪装成实机记录。“预期观察”表示根据命令语义应检查的现象。用户名、进程号、设备名和日期不会固定。本版已在目标虚拟机执行按章节组织的运行测试，并据此修正手册包、JDK 选择与 SSH 转发前提。预期失败、测试适配和未执行项

分别记录；重启恢复按用户要求暂不验证。各章分别说明已验证的操作、实测条件与未执行项目。

验证状态 | 已在 openEuler 24.03 LTS SP4 x86_64 完成主线运行测试，覆盖普通用户文件与权限、脚本、Java、systemd、SSH 隧道、升级与回滚。Spring Boot 构建使用独立镜像配置。未执行的项目为全新 ISO 安装、普通用户带密码的 `sudo` 交互，以及按作者要求暂缓的重启恢复验收。

0.3 给记忆留一次工作机会

0.3.1 四次短复习

完成一节后，十分钟内关掉示例，重做最短实验。一天后只看任务描述写命令。三天后给文件换一个名字，检查自己能否处理空格、路径和错误。七天后把本周学到的命令用于一个新任务，例如整理实验目录并打包备份。这些时间是便于安排的学习建议，不是保证记忆效果的固定公式。

每章的“不要往前翻”要求先解释，“命令填空”检查命令结构，“看需求写命令”要求自己组合，“看命令猜结果”帮助发现误解，“找错误”训练诊断，“无提示实操”检验能否独立完成。答案统一放在全书后部，先保留自己的答案再核对。

0.3.2 第一次学习记录

准备一页记录，写下系统版本、日期、今天完成的任务和一个仍不确定的问题。记录“权限不够，后来用了 `sudo`”过于粗略。应写清哪个文件、哪个用户、哪一项权限让操作被拒绝。第 8 章将提供检查方法。

0.4 本章练习

- 不要往前翻 | 命令没有输出时，怎样判断它是否完成了目标？
- 命令填空 | 打印当前位置使用 _____。
- 看需求写命令 | 创建家目录中的 linux-lab 并进入。
- 看命令猜结果 | 连续执行两次 `mkdir -p ~/linux-lab`，第二次会因目录已经存在而失败吗？
- 找错误 | 有人把 `cp [选项] <源> <目标>` 原样输入终端，问题在哪里？
- 无提示实操 | 重新登录，以普通用户进入实验目录，记录实际路径。

本章新命令与英文关联为 `mkdir / make directory`、`cd / change directory`、`pwd / print working directory`。`cp` 的第一次完整实验见第 4 章。复习时以自己的目录为准。

资料依据见来源 S03、S04。答案见全书“练习答案”第 0 章。

第 1 章 Linux 与 openEuler

本章任务 | 记录发行版、架构与内核，说明它们各自是什么。

1.1 让一台虚拟机能够工作

1.1.1 内核与发行版各做什么

打开电脑后，编辑器能读文件，浏览器能联网，多个程序能同时运行。它们需要一个共同的管理者分配处理器时间、内存和设备访问。这个核心部分叫内核，英文为 kernel。Linux 是一种开源内核，也常被用来泛指以它为基础整个操作系统。

单独的内核还不能提供本书全部学习环境。发行版把内核、常用工具、软件包管理系统和安装程序组合起来，并负责版本更新。openEuler 是本书使用的发行版。GNU 项目提供了许多用户空间工具，包括 Bash 和常见文件处理工具。GNU 的名称是递归缩写 GNU's Not Unix，了解这个来源即可，无须背历史年表。

应用程序通常运行在用户态，内核在内核态中执行需要更高特权的工作。程序通过系统调用请求内核读文件或建立网络连接。root 是用户身份，内核态是处理器执行权限层次，二者不能混为一谈。一个以 root 身份运行的普通程序仍主要运行在用户态。

图 1-1 从应用到硬件

应用程序

Bash 与常用工具



系统调用

Linux 内核



硬件

CPU / 内存 / 磁盘

1.1.2 确认要下载的版本

本书固定 openEuler 24.03 LTS SP4，不跟随“最新版”自动变更。LTS 表示 Long Term Support，长期支持；SP 是维护版本标识的一部分。核对版本时要保留 SP4，不能只记录 24.03。

从 openEuler 官方下载页进入对应版本与 x86_64 架构目录，选择标准离线安装镜像，并取得对应校验信息。ISO 是安装光盘内容的镜像文件。SHA256 是一种计算文件摘要的方法，下载后计算本地摘要并与官方值逐字符比较，能发现下载损坏或内容变化。摘要来源本身也应可信，不要用同一个陌生网盘提供的文本证明镜像可信。[S01、S02]

1.2 完成安装与第一次登录

1.2.1 新建专用虚拟机

在自己已有的虚拟化软件中，新建一台 64 位 Linux 虚拟机。作为本书练习的起始配置，可分配 2 个虚拟处理器、8 GiB 内存和 120 GiB 动态分配虚拟磁盘。这是学习配置建议，实际资源需结合宿主机余量与官方安装要求调整。动态分配表示宿主机文件随写入增长，并不意味着虚拟机永远不占空间。

挂载 ISO，启动安装程序。依次检查语言、时区、软件选择、安装目标和网络。选择基础命令行环境即可。安装目标只选择为本实验新建的虚拟磁盘。在确认写盘前核对容量，不把宿主机真实磁盘传给安装程序。保持默认分区方案有助于减少第一次安装的变量，第 15 章再理解它。

网络先采用虚拟化软件的 NAT 模式。NAT 让虚拟机借助宿主机的网络出口访问外部；从宿主机主动连接虚拟机可能还需要端口转发或另一张仅主机网络适配器，第 13 章会展开。

1.2.2 用户和管理员

创建普通账户，例如 alex。root 是系统超级用户，家目录通常是 /root；/home/alex 是普通用户 alex 的家目录。给账户设置自己的密码，输入密码时终端不回显星号也可能是正常行为。不要把真实密码写进学习记录。

安装器若提供管理员选项，可给自己创建的实验账户设置管理资格；不同安装选择的 sudo 策略仍需核实。sudo 按策略临时以另一身份执行命令，通常用于 root。它不会因为出现在命令前面就自动获得授权。普通实验不依赖 sudo。安装完成后弹出镜像并重启，确认从虚拟磁盘启动。

验证范围 | 已确认作者现有虚拟机为 openEuler 24.03 LTS SP4 x86_64，并在该环境完成后续主线运行测试。本轮未重新运行 ISO 安装器，也未复现普通账户输入密码取得 `sudo` 的交互；安装页面、安装时的软件选择和该交互流程单独保留为未验收项。

1.3 证明自己进入了哪台系统

1.3.1 三条只读命令

```
BASH
cat /etc/os-release
uname -m
uname -r
```

`cat` 关联 `concatenate`，原意是把输入连接起来输出，这里用于读取一个文本文件。`/etc/os-release` 保存发行版标识。`uname` 关联 Unix name，用于查看系统信息；`-m` 查看机器硬件名称，`-r` 查看内核发布版本。观察时分别记录发行版、架构和内核，别把内核版本当成 openEuler 的发行版本。

如果 `uname -m` 不是 `x86_64`，请回头确认镜像和虚拟机架构。某些 ARM 电脑使用的是 AArch64 虚拟机，很多命令仍相同，但这与本书固定的验证环境不同。先记下差异，再决定是否重建符合本书条件的实验机。

1.3.2 保存一个干净起点

在虚拟机软件里创建安装完成后的快照，并给它写一段说明，记录用户、网络方式和系统版本。后面实验出错，可以比较更改前后的状态。快照恢复会撤销其后磁盘上的实验文件，因此重要笔记另存一份。

1.4 本章练习

- 不要往前翻 | 内核、发行版和 Bash 分别负责什么？
- 命令填空 | 查看处理器架构使用 `uname _____`。
- 看需求写命令 | 读取发行版标识，并单独查询内核版本。
- 看命令猜结果 | `uname -r` 会告诉你账户密码吗？它输出哪一类信息？
- 找错误 | “我是 root，所以我的编辑器一直在内核态运行”，错在哪里？
- 无提示实操 | 记录自己的发行版、SP 标识、架构和网络方式，制作干净快照。

本章新命令为 `cat / concatenate`、`uname / Unix name`。保留第 0 章的实验目录。明天登录后，先不用笔记找回它。

资料依据见 S01、S02、S04、S12。答案见第 1 章答案。

第 2 章 Terminal、Shell 与 Bash

本章任务 | 输出一行文字，查到帮助，并独立退出分页器。

2.1 光标后面发生了什么

2.1.1 一个窗口，三层工作

你在窗口中输入 `pwd`，按回车后得到一个目录。显示字符、接收按键的界面叫终端，英文 `terminal`。解释命令语法、组织程序运行的程序叫 Shell。`Bash` 是一种 Shell，名字来自 `Bourne Again SHell`，与早期 `Bourne Shell` 有历史联系。[S03]

终端窗口可以运行 `Bash`，也可以运行别的 Shell。`Windows` 的 `PowerShell` 窗口不会因为长得像 `Linux` 终端就支持本书全部命令。请在 `openEuler` 的控制台或已登录 `openEuler` 的远程终端中操作。

图 2-1 一条命令的工作过程

终端

接收键盘输入



Bash

解析与展开参数



程序与内核

执行并返回结果

2.1.2 先试一个容易看清结果的命令

```
BASH
```

```
printf '%s\n' 'Hello, openEuler'
```

`printf` 关联 `formatted printing`，即按格式输出。第一个参数 `'%s\n'` 是格式，`%s` 放入一段字符串，`\n` 输出换行。第二个参数是要输出的文字。两侧的单引号告诉

Bash 把里面的空格和符号作为文字的一部分。终端应显示一行 Hello, openEuler。这是由输入决定的教学预期，不是保存的目标系统终端记录。

与文字类似，命令也需要明确边界。printf '%s\n' 'two words' 传入一个含空格的文本参数；把单引号删掉，two 和 words 成为两个参数，该格式会分别用于它们。这为第 4 章带空格的文件名作准备。

2.2 分清命令来自哪里

2.2.1 内建命令与外部程序

BASH

```
type cd
type pwd
type ls
```

type 的普通词义是类型，这里显示 Bash 会怎样解释一个名字。cd 通常是 Bash 内建命令，即解释器本身实现的命令。ls 通常来自独立可执行文件，也可能被配置成别名。pwd 既可以由 Bash 内建实现，也可能存在外部程序。以自己的 type 输出为准。

cd 需要改变当前 Shell 的工作目录，所以它必须影响当前 Shell 自己。外部程序在自己的进程中改变目录，不会让父 Shell 自动跟着改变。第 10 章会解释进程关系。

2.2.2 请求帮助

BASH

```
help cd
ls --help
man ls
```

help 是 Bash 的内建帮助命令，适用于 cd 这样的内建命令。--help 是许多外部命令提供的帮助选项。man 来自 manual，打开手册页。手册常由分页器显示，按 q 退出，空格翻页，输入 / 后接词语可搜索。若 man 尚未安装，先使用 --help；第 9 章再通过仓库查询安装，不能把缺少文档程序误判为整个系统损坏。

实测提示 | 有 man 命令，也可能没有某个程序的手册。本次 SP4 环境中，man ls 最初提示 No manual entry for ls；查询后确认 ls 的手册由 coreutils-help 提供。先用 ls --help 继续学习，到第 9 章补齐文档包。

2.3 少敲一点，少错一点

2.3.1 补全和历史

输入一部分路径后按 Tab，Bash 常能补齐唯一匹配项。出现多个候选时，继续输入更多字符。上、下方向键可以找回近期命令，先检查再回车。不要在历史命令里保存密码、访问令牌或数据库连接秘密。

Ctrl+C 通常请求中断当前前台任务。Ctrl+D 在空输入处表示输入结束，交互式 Shell 可能因此退出。它们不是同一个“取消”键。如果输入引号后忘了闭合，终端可能换到续行提示符；此时 Ctrl+C 可以放弃这一条未完成输入。

2.3.2 有意拼错一次

```
BASH
pwd
```

预期会出现包含 command not found 含义的错误。它说明 Shell 没能把 pwd 解析为可执行的命令。先对照拼写，再用 type 查询。以后还要检查 PATH，第 16 章会解释。不要看到这个错误就立刻下载来源不明的脚本。

2.4 本章练习

- 不要往前翻 | 终端、Shell、Bash 是怎样的关系？
- 命令填空 | 查询 cd 的内建帮助使用 _____ cd。
- 看需求写命令 | 输出一行包含空格的文字 Linux lab。
- 看命令猜结果 | printf '%s\n' alpha beta 输出几行？
- 找错误 | 把 pwd 写成 pwdd 后应先检查什么？
- 无提示实操 | 查询 cd、pwd、ls 的类型，打开一份帮助并独立退出。

本章新命令为 printf / formatted printing、type / 类型查询、help / 帮助、man / manual。ls 的名称与详细实验在第 4 章说明。今天的十分钟回忆可以只做一次文字输出和一次帮助查询。

资料依据见 S03、S04。答案见第 2 章答案。

第 3 章 Linux 文件系统与路径

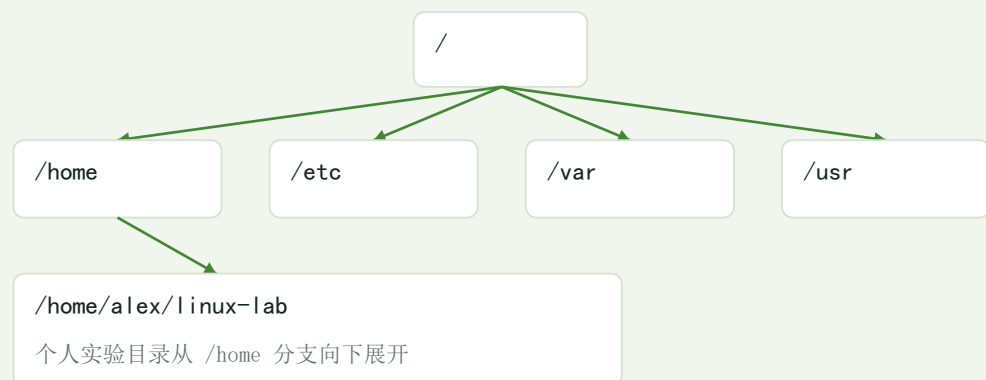
本章任务 | 从两个不同起点找到同一个目录，解释路径为何不同。

3.1 我现在在哪里

3.1.1 一棵从斜杠开始的目录树

文件要有名称，也要有位置。Linux 用目录组织位置，目录里还能包含目录。整个命名空间从 / 开始，这个位置叫根目录。/home、/etc 和 /var 都从这棵树向下展开。文件系统既可以指组织与保存文件的技术，也常用于讨论这棵可访问的目录树；磁盘与挂载的关系留到第 15 章。

图 3-1 从根目录展开



/etc 主要保存系统配置，/var 保存许多会变化的数据和日志，/usr 包含大量系统提供的程序与资源。/tmp 用于临时文件，其清理策略由系统决定，不适合保存长期作业。/root 是 root 用户的家目录，不能把它与 / 画等号。

3.1.2 工作目录是相对路径的起点

```
BASH
cd ~/linux-lab
mkdir -p paths/notes
cd paths
pwd
cd notes
pwd
```

第一行回到统一工作区。`mkdir -p paths/notes` 在其中建立两层目录。执行 `cd notes` 时，Bash 从当前的 `paths` 目录查找 `notes`。这个隐含起点叫当前工作目录。最后一次 `pwd` 应比前一次多一个 `/notes`。

3.2 同一个文件可以有几种写法

3.2.1 绝对路径与相对路径

以 `/` 开头的路径是绝对路径，从根目录开始找。相对路径从当前工作目录开始找。假如当前目录是 `/home/alex/linux-lab`，`paths/notes` 就是相对路径。此处的 `alex` 是示例用户名，你的家目录可能不同。

```
BASH
cd ~/linux-lab/paths/notes
cd ..
pwd
cd .
pwd
cd ~
pwd
```

`..` 表示上一级目录，`.` 表示当前位置。`cd ..` 后回到 `paths`，`cd .` 不改变位置。`~` 是 Shell 的家目录展开语法，在这里简化了输入。`/` 是系统根目录，`~` 随当前用户变化。不要把 `~` 写在单引号中期待它仍然展开。

图 3-2 同一个位置的两种写法

当前目录

`/home/alex/linux-lab`



相对路径

`paths/notes`



最终位置

`/home/alex/linux-lab/paths/notes`

3.2.2 走错路时怎样检查

BASH

```
cd ~/linux-lab/paths
cd missing
pwd
```

missing 尚未创建，预期出现 No such file or directory 一类报错。失败的 cd 不会自动进入别处。此时 pwd 仍应显示 paths。继续操作前先确认位置，尤其不要因为一条 cd 失败就让后续删除命令在错误目录执行。

3.3 大小写与空格都属于名字

3.3.1 让 Shell 把名字看完整

BASH

```
cd ~/linux-lab/paths
mkdir 'reading notes'
cd 'reading notes'
pwd
```

单引号保护中间的空格，mkdir 得到一个名称参数。若写成 mkdir reading notes，它会创建两个目录。常见 Linux 文件系统区分大小写，Notes 与 notes 可能是两个位置。不要依赖 Windows 文件管理习惯推断结果。

3.3.2 返回家目录再重来

cd 不带参数通常也会回到家目录，cd - 可以切回上一个工作目录。前者是 home 的快捷入口，后者依赖此前的切换记录；刚启动 Shell 时不一定已有可用的上一个目录。初学阶段先多用完整路径与 pwd，等位置感建立后再减少查询。

3.4 本章练习

- 不要往前翻 | `/`、`/root`、`~` 有什么区别？
- 命令填空 | 返回上一级使用 `cd` _____。
- 看需求写命令 | 从任何位置进入家目录中的 `linux-lab/paths/notes`。
- 看命令猜结果 | 在 `paths` 中执行 `cd notes` 后再执行 `cd ..`，回到哪里？
- 找错误 | `cd reading notes` 为什么不能按预想进入一个含空格的目录？
- 无提示实操 | 建立 `project/src` 和 `project/docs`，进入 `src`，用相对路径进入 `docs`，再用绝对路径思想解释自己的位置。

本章复用 `mkdir`、`cd`、`pwd`，无须新增命令。一天后换一组目录名再做任务，保留实际路径记录。

资料依据见 S03、S04。答案见第 3 章答案。

第 4 章 文件与目录操作

本章任务 | 完成一次复制、移动和链接实验，能指出文件最终在哪里。

4.1 为一份文件建立副本

4.1.1 创建并看见它

```
BASH
mkdir -p ~/linux-lab/files
cd ~/linux-lab/files
touch hello.txt
ls -l
```

`touch` 的英文意思是触碰，它的主要用途是更新文件的访问时间或修改时间。目标不存在且父目录可写时，它会创建空文件；已有内容不会因此清空。[S04a] `ls` 可与 `list` 关联，列出目录内容。`-l` 选择长格式。

长格式通常依次包含类型与权限、硬链接数、所有者、所属组、大小、时间和名称。普通空文件的大小应为 0 字节。第一列以 `-` 开头通常表示普通文件，以 `d` 开头表示目录。各列的具体值取决于系统，暂时不用背权限串，第 8 章会逐位拆解。

4.1.2 复制时源在前、目标在后

```
BASH
cp hello.txt backup.txt
ls -l
```

`cp` 是 `copy` 的缩写。`hello.txt` 是被读取的源，`backup.txt` 是写出的目标。复制后两个名称都应存在，之后修改其中一份通常不会改变另一份的内容。目标如果已经存在，普通 `cp` 可能覆盖它；练习前检查文件名，想在覆盖前确认可用 `cp -i`。

图 4-1 复制保留源文件

复制前

hello.txt

**cp hello.txt backup.txt**

读取源并写出目标

**复制后**

hello.txt 与 backup.txt

4.2 改名字与搬位置

4.2.1 mv 的两种常见使用方式

BASH

```
mv backup.txt draft.txt
mkdir archive
mv draft.txt archive/
ls archive
```

mv 关联 move。第一次把文件名称从 backup.txt 改为 draft.txt，第二次把它移入 archive 目录。archive/ 后的斜杠表明期望目标是目录，可以帮助发现把目录误写成文件的情况。同一文件系统内的移动常能直接调整目录项，跨文件系统移动则可能需要复制数据再删除源，因此不要把所有 mv 都理解成瞬间完成。

4.2.2 复制目录需要明确递归

BASH

```
cp -r archive archive-copy
ls archive-copy
```

-r 与 --recursive 在 GNU cp 中表示递归，把目录内容逐层复制。这里 archive-copy 事先不存在，所以成为副本根目录。如果它已经是目录，目标层次可能多出 archive 这一层。重做实验时先查看现有目录，再预测结果。[S04b]

4.3 删除之前确认对象

4.3.1 从一个文件开始

```
BASH
pwd
ls -l hello.txt
rm -i hello.txt
```

rm 关联 remove，移除名称。-i 请求逐项确认，确认提示的具体语言因区域设置而异。只在确认目标是本章空文件后同意。命令行删除通常没有桌面回收站，不能把 -i 当作备份。不要在初学时用递归强制删除整理整个实验目录。

rmdir 关联 remove directory，只删除空目录。运行 rmdir archive 时，如果里面还有 draft.txt，预期会得到 Directory not empty 一类错误。这个拒绝能帮助你发现目录中仍有文件。先列出、逐项处理自己创建的文件，再删除空目录。

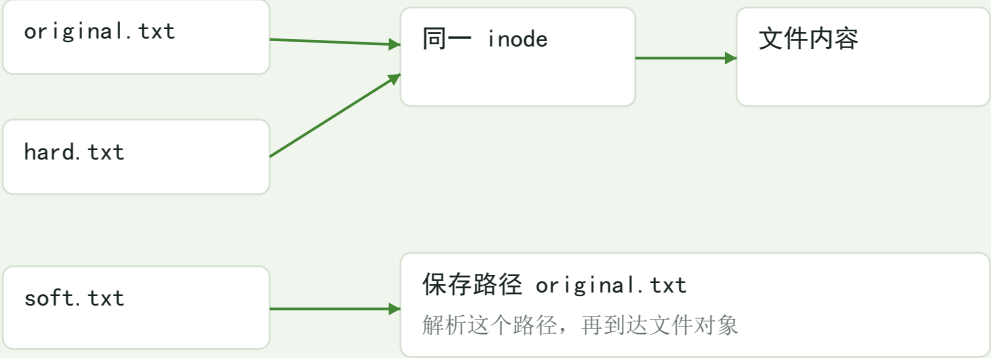
4.3.2 两种链接

```
BASH
printf '%s\n' 'version one' > original.txt
ln original.txt hard.txt
ln -s original.txt soft.txt
ls -li original.txt hard.txt soft.txt
```

这一处先使用 > 把文字保存到文件，若同名文件存在会覆盖；第 17 章完整解释。ln 与 link 关联，默认创建硬链接；-s 表示 symbolic，创建符号链接。-i 让 ls 显示 inode 编号，inode 是文件系统管理文件元数据的对象，名称通过目录项关联到它。

硬链接为同一个文件对象增加名字，通常不能跨文件系统，普通用户也不应给目录建立硬链接。软链接保存目标路径，可指向目录，也可跨文件系统。相对软链接路径以链接所在目录为起点解析。删掉 original.txt 后，hard.txt 仍关联原对象，soft.txt 的路径则可能失效。文件在最后一个硬链接被移除、且不再被进程持有等条件满足后，空间才真正得到释放。

图 4-2 名称与文件对象



4.4 本章练习

- 不要往前翻 | `touch` 已有文件会清空内容吗？
- 命令填空 | 复制目录使用 `cp _____ source target`。
- 看需求写命令 | 把本章已创建的 `original.txt` 复制为 `original.bak`，再把副本移入 `archive`。
- 看命令猜结果 | 对同一个文件创建两个硬链接，修改其中一名所指内容后另一名会看到什么？
- 找错误 | `rmdir` 报 `Directory not empty` 时，直接加不存在的“强制参数”合适吗？
- 无提示实操 | 在新目录中创建一份有内容的笔记，复制、重命名、归档，检查两份内容，最后只删除你指定的副本。

本章新命令与英文关联为 `touch` / 触碰并更新时间、`ls` / `list`、`cp` / `copy`、`mv` / `move`、`rm` / `remove`、`rmdir` / `remove directory`、`ln` / `link`。七天后的复习任务可以是整理自己的实验笔记。

资料依据见 S04、S04a、S04b、S12。答案见第 4 章答案。

第 5 章 查看与处理文本

本章任务 | 从一份日志中取出需要的行，并保留原始数据。

5.1 文件有内容以后怎样阅读

5.1.1 准备一份小日志

日志是程序按时间或事件写出的记录。我们先手工制造一份教学数据，练习查看方法，不把它当作某个真实服务的运行记录。

BASH

```
mkdir -p ~/linux-lab/text
cd ~/linux-lab/text
printf '%s\n' 'INFO start' 'INFO ready' 'WARN slow' \
  'ERROR timeout' 'INFO stop' > app.log
cat app.log
```

行尾的反斜杠表示这条命令在下一物理行继续，反斜杠后面不要再加空格。printf 按格式输出五个参数，> 把内容保存到 app.log。这份文件较短，可以用 cat 一次读完。文件很大时，连续滚屏很难定位，这时换成分页阅读。

5.1.2 less 让阅读可以停下来

BASH

```
less app.log
```

less 是分页器，其名字与早期 more 分页工具有联系；这里把它记成可以前后移动的阅读工具即可。按上下方向键移动，空格向后翻一屏，b 向前翻一屏，/ERROR 搜索 ERROR，n 找下一个匹配，q 退出。阅读工具一般不修改文件，退出后文件仍保持原样。最小安装若没有 less，先用 cat，包管理方法见第 9 章。

5.2 只看需要的几行

5.2.1 头部和尾部

BASH

```
head -n 2 app.log
tail -n 2 app.log
wc -l app.log
```

`head` 和 `tail` 分别是头部、尾部的英文词。`-n 2` 指定两行。第一条应选出前两行，第二条应选出最后两行。`wc` 来源可与 `word count` 关联，`-l` 统计换行符数量；本例每行都以换行结束，所以结果为五。如果文件最后一段没有换行，肉眼看到的行数可能与 `wc -l` 不同。

5.2.2 跟随追加内容

在终端 A 中运行 `tail -f app.log`，`-f` 表示 `follow`，等待文件的新内容。在终端 B 中进入同一目录，运行下面一行。

BASH

```
printf '%s\n' 'INFO another event' >> app.log
```

`>>` 追加而不清空旧内容。回到终端 A 应能看见新增行。按 `Ctrl+C` 结束跟随，并不删除日志。如果程序会把旧日志改名后新建同名文件，GNU `tail` 的 `-F` 还能结合按名称跟随与重试；普通 `-f` 与 `-F` 在日志轮转时的差异应结合实际程序验证。

5.3 把重复内容整理出来

5.3.1 顺序决定去重结果

BASH

```
printf '%s\n' pear apple pear apple > fruit.txt
sort fruit.txt
sort fruit.txt > sorted.txt
uniq -c sorted.txt
```

`sort` 就是排序，默认按当前区域设置的排序规则处理整行。它把结果写到标准输出，不会自动修改源文件。`uniq` 与 `unique` 关联，只合并相邻重复行；`-c` 显示相邻组的出现次数。因此我们先排序，再用 `uniq`。输出中的顺序可能随区域设置变化，若实验要求固定字节顺序，可在命令前写 `LC_ALL=C`。

5.3.2 切出简单字段

BASH

```
printf '%s\n' 'alex:student' 'sam:teacher' > roles.txt  
cut -d ':' -f 1 roles.txt
```

cut 的意思是切取，-d 指定分隔符，-f 1 选择第一字段。这里每行用一个冒号分开名称和角色。预期输出 alex 与 sam。复杂 CSV 可能含引号和字段内逗号，不能直接把 cut -d ',' 当作完整 CSV 解析器。

5.4 本章练习

- 不要往前翻 | cat 与 less 适合怎样的阅读场景？
- 命令填空 | 读取最后三行使用 tail _____ 3 app.log。
- 看需求写命令 | 查看日志前两行并统计换行符数量。
- 看命令猜结果 | fruit.txt 未排序直接交给 uniq，隔行重复的 pear 会全部合并吗？
- 找错误 | sort fruit.txt > fruit.txt 为什么可能把输入清空？先保留这个问题，到第 17 章再解释打开文件的时机。
- 无提示实操 | 建立一份六行名单，把排序结果保存为新文件，统计每个相邻重复组的数量，保留原始文件。

本章新命令为 less / 分页器名称、head / 头、tail / 尾、wc / word count、sort / 排序、uniq / unique、cut / 切取。复习时仍使用第 4 章的 cp，先保留数据副本。

资料依据见 S04、S03。答案见第 5 章答案。

第 6 章 Vim

本章任务 | 编辑三行笔记，保存后在终端验证实际内容。

6.1 能进去，也能出来

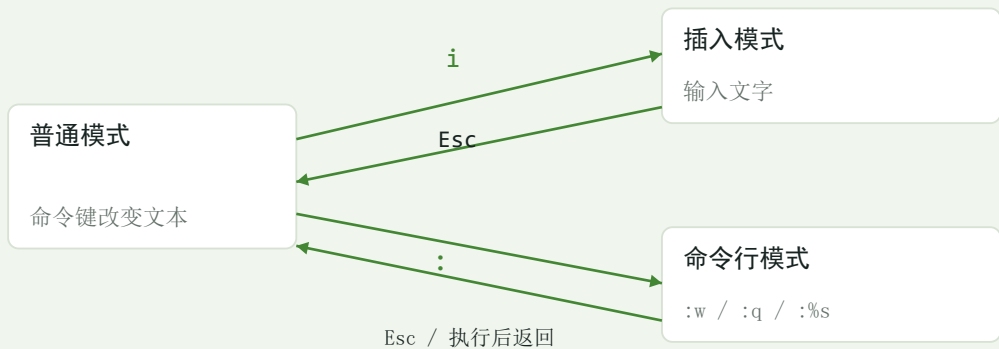
6.1.1 从一份自己的笔记开始

服务器上不一定有图形编辑器。Vim 可以直接在终端中编辑文本，名字来自 Vi Improved，即对 vi 编辑器的改进。先查询 `type vim`。如果不存在，暂用前面的 `printf` 创建文件，到第 9 章按仓库查询方法安装所需编辑器后再回来完成本章。

```
BASH
mkdir -p ~/linux-lab/editor
cd ~/linux-lab/editor
vim note.txt
```

Vim 启动后默认处于普通模式。这个模式中按键通常是编辑命令。按 `i` 进入插入模式，再输入两行文字。按 `Esc` 回到普通模式，输入 `:wq` 并回车，保存并退出。`w` 与 `write` 关联，`q` 与 `quit` 关联。键序列不带反引号，也不要把 `:wq` 输入到 Bash 提示符后。

图 6-1 输入之前先认模式



提示 | 键盘输入变成正文时，先按 `Esc` 确认回到普通模式，再输入编辑命令。退出之前用 `:w` 保存，退出之后用 `cat` 核对磁盘内容。

6.1.2 如果只是误打开

按 `Esc`，再输入 `:q` 回车可退出未修改的文件。如果修改过但不想保存，使用 `:q!`；感叹号表示放弃相关保护检查，会丢失本次未保存的修改。已写入磁盘的历史版本不会因此恢复。想保留修改但暂不退出，使用 `:w`。

6.2 让修改动作可控

6.2.1 移动、删除与撤销

重新打开 `note.txt`。普通模式中可以用方向键移动，也可以用 `h`、`j`、`k`、`l` 分别向左、下、上、右。先把精力放在模式上，不必强迫自己马上放弃方向键。

把光标放到一行上，按 `dd` 删除这一行。按 `u` 撤销，`Ctrl+R` 重做。按 `yy` 复制当前行，`p` 放到当前位置之后。数字通常可以修饰动作，例如 `3j` 向下移动三行。不要在插入模式中尝试这些命令，否则字母会变成正文。

6.2.2 查找比逐行移动更可靠

普通模式输入 `/Linux` 并回车，查找 `Linux`。`n` 跳到下一个匹配，`N` 反向查找。输入 `:set number` 显示行号，这只影响当前编辑显示。发现第几行的配置有问题后，可以输入 `:12` 跳到第 12 行。

要把全文中的 `lab` 改为 `practice`，可输入下面的 Vim 命令。

```
VIM
:%s/lab/practice/gc
```

`%` 代表全部行，`s` 关联 `substitute`，替换匹配文本。`g` 让一行内多个匹配都参与，`c` 要求逐个确认。模式采用正则表达式，不等同于任意文字的原样搜索，第 18 章介绍基本规则。初次批量替换时保留 `c`，比不看范围就替换更容易发现错误。

6.3 报错往往与文件有关

6.3.1 保存失败先检查路径与权限

编辑器可以打开一个你能读、却不能写的文件。保存时出现只读或权限错误，应检查文件和父目录权限。对系统配置，应先退出、备份并明确修改目的；第 8 章解释权限，第 23 章给出专用配置操作。不要为了存一行练习而把整个编辑器长期以 `root` 身份运行。

6.3.2 交换文件提示

异常断线后，Vim 可能提示存在交换文件。也可能是另一终端仍在编辑同一个文件。先确认是否还有编辑会话，必要时按提示恢复，再比较恢复内容与磁盘文件。不要不看说明就删除交换文件，那里面可能保留尚未保存的工作。读懂提示之后选择恢复、只读打开或退出。

6.4 本章练习

- 不要往前翻 | 普通模式和插入模式分别解释按键为什么？
- 命令填空 | 保存退出使用 _____，放弃未保存改动退出使用 _____。
- 看需求写命令 | 显示行号并跳到第 5 行，写出 Vim 中要输入的命令。
- 看命令猜结果 | 在普通模式按 dd 后按 u，当前行会怎样？
- 找错误 | 在 Bash 输入 :wq 为什么无法关闭一个已经退出的 Vim？
- 无提示实操 | 写一份三行笔记，复制一行、删一行、撤销，再搜索一个词并保存。用 cat 验证磁盘内容。

本章新工具为 Vim / Vi IMproved。编辑结束后复用 cat 检查，三天后只看任务重做一次。

资料依据见 S05。答案见第 6 章答案。

第 7 章 用户与用户组

本章任务 | 确认有效用户和组，解释一次管理权限从哪里来。

7.1 程序究竟以谁的身份运行

7.1.1 看看当前账户

```
BASH
whoami
id
```

whoami 连起来就是 who am I，显示当前有效用户名称。id 是 identity 的词义关联，用数字与名称展示用户及所属组。UID 是 user identifier，用户标识；GID 是 group identifier，组标识。名字方便人阅读，系统在许多权限判断中使用数字。

一个用户通常有一个主组，还可以属于多个附加组。组为多个账户共享权限提供一种组织方式。把人加入一个组，并不会自动让所有文件对这个组开放，文件的所属组与权限仍需配合。

图 7-1 身份与组关系

用户

UID 标识账户



主组 + 附加组

多个 GID 参与权限判断



文件对象

所有者 / 所属组 / 权限

7.1.2 查询账户数据库

```
BASH
getent passwd "$(id -un)"
getent group
```

getent 可理解为 get entries，按系统配置查询名称服务数据库条目。passwd 在这里是数据库名，不是在改密码。`$(id -un)` 先查询当前有效用户名，再把结果作为一个参数交给 getent。`-u` 查询有效用户 ID，`-n` 改为显示名字。环境变量 USER 可能因切换身份或人为赋值而保留旧值，查询当前身份时优先使用 id。passwd 条目通常包含用户名、密码占位符、UID、GID、说明、家目录和登录 Shell，字段以冒号分隔。密码散列通常放在受保护的 `/etc/shadow` 中，不应复制到学习笔记。

7.2 临时管理权限与切换身份

7.2.1 sudo 的范围

```
BASH
sudo -l
```

sudo 可以理解为以指定用户执行命令的工具，默认目标通常是 root。名称历史常与 superuser do 关联，但实际能力包含其他目标用户，因此本书不把它解释成只能“变成 root”。`-l` 列出当前策略允许的操作，可能要求输入当前用户自己的密码。

su 表示 substitute user，切换用户身份；`su -` 使用接近登录的环境。默认目标是 root，认证要求取决于策略，通常涉及目标账户密码。sudo 与 su 的授权机制和环境行为不同，不能只按“都能提权”互换。

7.2.2 没有授权时怎样继续

若普通用户不在 sudo 授权范围内，先使用虚拟机控制台与安装时建立的管理员入口。由已授权管理员检查本机策略，必要时通过 visudo 修改，visudo 会在保存时检查语法。不要直接向 `/etc/sudoers` 随意追加文本。把用户加入 wheel 是否获得 sudo 权限取决于启用的策略，不能只凭组名断言。

实机记录 | 本次目标机启用了 wheel 的 sudo 策略，普通测试账户未获授权时被拒绝；管理步骤由已有 root 连接执行。没有向测试账户授予免密 sudo。读者以带密码的普通账户完成 sudo 授权与交互登录，仍需按自己的安装策略验证。

7.3 创建一个可辨认的练习账户

7.3.1 先检查名称，再创建

只在自己的专用虚拟机中完成本节。labreader 是本书练习名，若已存在就改用另一个未使用的名字，并在后续命令保持一致。

BASH

```
getent passwd labreader
sudo useradd -m labreader
id labreader
```

第一条如果已有输出，停止创建并检查现有账户用途。useradd 由 user 与 add 组成，创建用户；-m 要求创建家目录。没有给该账户设置登录密码，因此不要把“账户创建成功”理解成“可以立即密码登录”。登录策略还受锁定状态与 Shell 等条件影响。

7.3.2 附加组不能误覆盖

groupadd 用于新增组，usermod 用于修改账户。下面创建练习组并把 labreader 追加进去，同样先确认名称未被使用。

BASH

```
getent group labteam
sudo groupadd labteam
sudo usermod -aG labteam labreader
id labreader
```

-G 设置附加组列表，-a 表示 append，追加。遗漏 -a 可能把其他附加组从列表中移除。组关系变化后，已有进程不会自动全部获得新组身份，通常需要新登录会话。实验结束后可以保留账户用于第 8 章观察，无须急着删除家目录。

7.4 本章练习

- 不要往前翻 | 主组与附加组有什么用途？
- 命令填空 | 查看自己的 UID 和组列表使用 _____。
- 看需求写命令 | 查询 labreader 的账户条目与组归属。
- 看命令猜结果 | `usermod -G team 用户名` 与加上 `-a` 后的操作有什么差异？
- 找错误 | “加入 wheel 就一定能用 sudo”，遗漏了什么条件？
- 无提示实操 | 在记录中列出普通用户、管理员、专用服务用户各需要怎样的权限，暂不创建服务用户，留给第 24 章。

本章新命令为 `whoami` / `who am I`、`id` / `identity`、`getent` / `get entries`、`sudo` / 按策略换身份执行、`su` / `substitute user`、`useradd` / `add user`、`groupadd` / `add group`、`usermod` / `modify user`、`visudo` / 安全编辑 `sudo` 策略。名称关联服务于理解，不代表所有名称都有一个唯一正式英文全称。

资料依据见 S06、S12。答案见第 7 章答案。

第 8 章 Linux 权限

本章任务 | 分别解释普通文件和目录的 `rw`x，不靠反复试 777。

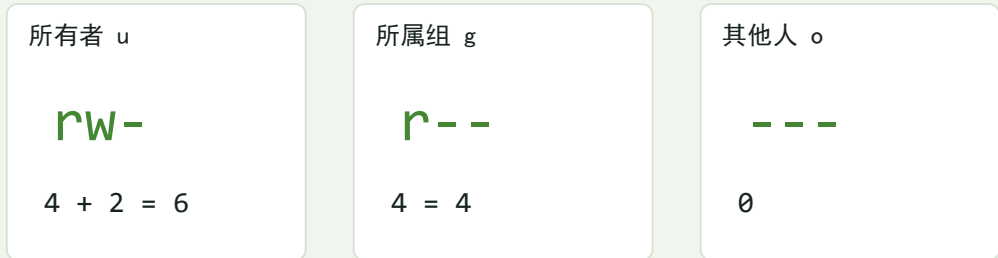
8.1 为什么自己创建的文件也可能不能写

8.1.1 看懂三组 `rw`x

```
BASH
mkdir -p ~/linux-lab/permissions
cd ~/linux-lab/permissions
printf '%s\n' 'private note' > note.txt
chmod 600 note.txt
ls -l note.txt
```

`chmod` 关联 `change mode`，改变权限模式。600 的含义稍后解释。先观察权限区域，普通文件类型标记 - 后面依次是所有者、所属组、其他人的三组权限。`r` 是 `read`，读；`w` 是 `write`，写；`x` 是 `execute`，执行。

图 8-1 把权限拆成三组



普通文件类型 - 与三组权限分开看。组合示例为 640。

对普通文件，`r` 允许读取内容，`w` 允许修改内容，`x` 允许尝试执行。文件有 `x` 不代表它一定是有效程序，也不保证解释器或加载器存在。系统首先根据身份选择适用权限类别，不能把所有者缺少的权限再随意从“其他人”补回来。ACL 等扩展权限机制会增加细节，本章先掌握基本模型。

8.1.2 数字来自三个位的组合

r 对应 4, w 对应 2, x 对应 1, 每组相加。6 是读写, 5 是读与执行, 7 是三者都有, 0 是都没有。600 表示所有者可读写, 组和其他人都没有权限。644 常用于允许别人读取的普通文件, 700 常用于只让自己访问的目录。权限应根据任务决定, 不要把这些数当作任何场景的固定答案。

8.2 目录权限与文件权限不同

8.2.1 能列名字不等于能读取内容

目录的 r 允许读取目录项列表, x 允许搜索或穿过这个目录, w 配合 x 通常允许新增、删除或重命名其中的目录项。读取一个文件通常需要父路径各级目录的 x 和文件自身的 r。

删除文件主要检查父目录权限, 文件本身只读不能阻止有权修改父目录项的用户删掉它。带 sticky bit 的共享目录还会增加删除限制, /tmp 经常使用这种机制。遇到删除失败, 不要只盯着文件自身的 w。

8.2.2 使用符号方式增加一项

BASH

```
chmod u-w note.txt
ls -l note.txt
chmod u+w note.txt
mkdir private
chmod 700 private
ls -ld private
```

u 是 user, 即所有者, g 是 group, o 是 others, a 表示全部类别。+ 增加权限, - 移除, = 设为指定权限。u-w 只移除所有者的写权限, u+w 把它加回。ls -d 显示目录本身的条目, 避免误看目录内部文件的权限。

8.3 所有者和默认权限

8.3.1 chown 改变的是归属

chown 关联 change owner, 用来更改所有者, 也能指定组。普通用户不能随意把文件交给其他用户。只有专门的管理任务才需要 sudo。第 24 章把服务目录交给专用账户时会给出具体命令, 当前无需改变实验目录归属。

递归选项 `-R` 会影响子目录中的大量对象。对 `chmod -R` 或 `chown -R`，必须事先确认绝对路径和内容。不要对整个家目录“试一个权限数字”。目录通常需要 `x`，数据文件往往不需要，一条数字递归套用可能同时破坏两类对象。

8.3.2 umask 从新建请求中去掉权限

```
BASH
umask
```

`umask` 可与 `user file creation mask` 关联，文件创建权限掩码。常见程序创建普通文件时请求 `666`，创建目录时请求 `777`，掩码按位移除权限。例如 `022` 常让这些请求变成 `644` 和 `755`。它不是简单十进制减法，而且程序可能主动请求更严格的权限；目录默认 `ACL` 也可能影响结果。修改 `umask` 不会追溯改变旧文件。

提示 | 权限实验必须用普通用户观察。`root` 可能越过部分普通文件权限检查；用 `root` 测试“没有读权限能否读取”，不能代表普通用户的结果。

8.4 权限报错的检查顺序

看到 `Permission denied`，先运行 `id` 确认身份，用 `pwd` 确认位置，再用 `ls -ld` 检查父目录、`ls -l` 检查文件。若基本权限正确，还应检查 `ACL`、只读挂载和安全策略。`Operation not permitted` 可能涉及能力限制、文件属性或具体系统调用，不等于“加一个 `w` 就好”。第 23 章将把这些检查用于服务部署。

8.5 本章练习

- 不要往前翻 | 文件的 `x` 与目录的 `x` 分别允许什么？
- 命令填空 | 仅给所有者增加执行权限使用 `chmod _____ script.sh`。
- 看需求写命令 | 让 `note.txt` 仅所有者可读写，让 `private` 仅所有者可访问。
- 看命令猜结果 | 文件模式 `444`，但父目录允许你写入和穿过，是否绝对无法删除该文件？
- 找错误 | 为了修复一个配置文件读取失败，把整棵目录改成 `777` 有何问题？
- 无提示实操 | 创建一个私有目录和笔记，证明你检查的是目录本身与文件本身；解释每个数字，不要只背答案。

本章新命令为 `chmod` / `change mode`、`chown` / `change owner`、`umask` / `creation mask`。复用 `id`、`ls`、`mkdir`，七天后重新从报错检查顺序开始回忆。

资料依据见 S04c、S06、S12。答案见第 8 章答案。

第 9 章 DNF 与 RPM

本章任务 | 从一个命令反查软件包，再判断是否需要安装。

9.1 一个命令不存在时该做什么

9.1.1 程序、包与仓库

输入 `vim` 后收到 `command not found`，先检查拼写和 `type vim`。确认确实没有后，再找它由哪个软件包提供。软件包把程序、文件清单、版本和依赖关系组织在一起。仓库是发布软件包及索引的位置；索引让管理工具知道有哪些包、适用架构和依赖。

DNF 是高层包管理器，名称沿用 Dandified YUM。RPM 是 RPM Package Manager，负责 RPM 包格式及已安装包数据库等工作。本书使用 DNF 从配置的仓库解析依赖，用 RPM 查询系统中的包与文件归属，不把两者看成互不相干的安装方式。

9.1.2 先看系统提供了什么

```
BASH
dnf --version
dnf repolist
dnf search vim
dnf provides '*/vim'
```

`repolist` 列出仓库，`search` 搜索包信息，`provides` 查询提供某项能力或文件的包。’`*/vim`’ 用单引号保护，避免 Bash 在当前目录先展开星号。这些查询先以普通用户执行；若本机策略或缓存权限拒绝，再由管理员核查原因。查询可能下载元数据，因此需要可用网络；报 DNS、证书或镜像访问错误时，先诊断网络和时间，不要关闭签名检查。

9.2 完成一次可审查的安装

9.2.1 查看候选包

在本机查询结果中确认编辑器包的准确名称，然后使用 `dnf info` 查询详情。以下以 `vim-enhanced` 为候选示例，只有查询确认它来自目标版本的可信仓库后才安装。

```
BASH
dnf info vim-enhanced
sudo dnf install vim-enhanced
```

安装需要写系统目录和包数据库，所以使用 `sudo`。DNF 会给出拟安装、升级或移除的包及空间信息。读过交易摘要后再确认。教程不加 `-y`，让初学者有一次检查依赖变化的机会。若提示找不到该包，回到 `provides` 的实际结果，不能换一个网上猜来的包名继续。

实机记录 | 本次 SP4 x86_64 环境已查询并安装 `vim-enhanced`、`coreutils-help`、`git`、`java-21-openjdk-devel` 和 `maven`，保留了交易记录。仓库会更新，读者仍需检查自己看到的版本、依赖和来源。

9.2.2 安装后的证据

BASH

```
type vim
rpm -q vim-enhanced
rpm -ql vim-enhanced
```

`rpm -q` 中 `q` 是 `query`，查询一个已安装包；`-ql` 继续列出包里的文件。要查询 `/usr/bin/vim` 由谁提供，可用 `rpm -qf /usr/bin/vim`，`f` 对应 `file`。这能把终端中用到的程序关联回安装记录。

9.2.3 手册程序与文档包分开检查

若 `man` 已安装，`man ls` 却找不到条目，可以查询谁提供这份文档。

BASH

```
dnf provides '*/man1/ls.1*'
dnf info coreutils-help
```

本次查询结果指向 `coreutils-help`。核对交易后安装，再打开手册。

BASH

```
sudo dnf install coreutils-help
man ls
```

不要仅凭手册缺失就重装 `coreutils` 主程序。查询、确认提供者、安装、验证的顺序，同样适用于其他缺少的帮助文件。

9.3 更新、移除与来源

9.3.1 变更前保留可恢复状态

`dnf check-update` 查询更新，退出状态 100 通常表示存在可用更新，不应被自动脚本一律当成故障。真正升级会改变系统，需要先保存工作与快照，检查交易内容。服务器应安排维护窗口，再确定是否需要重启进程或系统。安装历史便于追溯，不能保证任何更新都能完整回退，旧包可能已不可获取，数据格式也可能已经变化。

移除软件用 `dnf remove` 后接准确包名，先读依赖移除列表。不要练习移除 Bash、包管理器、SSH 或系统核心组件。对自己安装的可选工具，理解影响后再操作即可。

提示 | 退出状态 0 表示没有更新，100 表示有更新，1 表示错误。写巡检脚本时，要区分“发现更新”和“查询失败”，同时保留诊断输出。[S07]

9.3.2 不混用不匹配的软件源

仓库配置常见于 `/etc/yum.repos.d/` 中的 `.repo` 文件，但应通过本机配置确认。修改前先读取和备份。不要把 Ubuntu 的 `apt` 命令或旧 CentOS 源地址当作 openEuler 24.03 SP4 的安装方法。把别的发行版 RPM 强行装进来，也可能造成库依赖冲突。

从本地 RPM 安装时，DNF 仍可处理依赖。发布方签名、可信下载路径与包来源都要核对。遇到签名问题应调查原因，不能为了让命令通过就写入 `gpgcheck=0`。

9.4 本章练习

- 不要往前翻 | 仓库索引、RPM 包和可执行文件有什么联系？
- 命令填空 | 查询一个文件属于哪个已安装包使用 `rpm _____` / 路径。
- 看需求写命令 | 搜索 Vim，并查询哪个包提供以 `/vim` 结尾的程序路径。
- 看命令猜结果 | `dnf check-update` 返回 100 是否一定表示网络失败？
- 找错误 | 为修复安装错误而混入未知 CentOS 仓库，会引入哪些新变量？
- 无提示实操 | 在虚拟机中为一个可选工具记录查询、交易摘要、安装后版本和文件归属。暂不做全系统升级。

本章新命令为 `dnf` / Dandified YUM、`rpm` / RPM Package Manager。把实际包名写进学习记录，三天后从一个程序路径反查包名。

资料依据见 S06、S07、S08、S27。答案见第 9 章答案。

第 10 章 进程与任务

本章任务 | 启动并结束自己的一条等待任务，用 PID 核对对象。

10.1 文件中的程序怎样变成正在做事的进程

10.1.1 观察一个会等待的程序

程序文件保存指令，进程是程序运行时的一个实例。同一程序可以同时启动多次，拥有不同的进程。PID 是 process identifier，进程标识；PPID 是 parent process identifier，父进程标识。

```
BASH
sleep 120 &
lab_pid=$!
printf '%s\n' "$lab_pid"
ps -p "$lab_pid" -o pid,ppid,stat,comm
```

sleep 的意思是等待，120 表示秒数。& 让命令在后台运行，不占住当前提示符。\$! 是 Bash 记录的最近一次后台异步任务的进程号，这里存入变量 lab_pid；等号两边不能有空格。ps 关联 process status，-p 选择 PID，-o 选择输出列。stat 是状态，comm 是命令名称。

图 10-1 进程关系与标识

Bash

父进程



sleep 120

子进程，获得自己的 PID



PPID

记录父进程的 PID

10.1.2 工作目录、环境与打开的文件

一个进程除了 PID，还带有用户身份、工作目录、环境变量和打开文件等状态。它的子进程通常继承其中一部分。这解释了为什么在某个目录启动程序，会影响它读取相对路径的位置。父进程退出后，子进程的后续处理还取决于会话、信号和服务管理方式，不能只记“父退出，子必死”这样过度简化的规则。

10.2 前台、后台与作业

10.2.1 当前 Shell 管理的作业

```
BASH
jobs
```

jobs 就是作业列表，显示当前 Shell 知道的后台或暂停任务。它不能列出整台机器的所有进程。作业编号与 PID 也不相同。fg 关联 foreground，把作业带到前台；bg 关联 background，让暂停作业继续在后台运行。

如果 jobs 显示刚才的 sleep 为当前作业，可以输入 fg 将其恢复到前台，然后用 Ctrl+C 中断。若 120 秒已经过去，作业可能早已结束，先看实际列表，不要机械使用固定编号。Ctrl+Z 通常暂停前台作业，暂停会保留进程并停止其执行，不能代替正常退出。

10.2.2 让进程结束

重新创建一条 sleep 任务，立即保存其 PID，再发送信号。

```
BASH
sleep 120 &
lab_pid=$!
ps -p "$lab_pid" -o pid,comm
kill -TERM "$lab_pid"
```

kill 的名称容易让人以为只有强制杀死一种操作，实际主要用途是发送信号。TERM 请求结束，程序通常有机会清理资源。KILL 则不能被捕获处理，所以不应作为第一选择。这里只对自己刚创建、已核对的 sleep 发信号。PID 会被复用，长时间保存的 PID 不能证明现在仍是原进程。

10.3 观察资源使用

10.3.1 从动态列表找线索

top 是动态进程监视工具，名称可直接记作上方列出的活跃任务视图，不必编造全称。运行 top 后观察 CPU、内存和进程列表，按 q 退出。一个进程 CPU 较高可以来自正常计算，内存“空闲很少”也可能包含可回收缓存。需要结合请求量、持续时间和系统是否出现延迟判断。

10.3.2 进程存在与服务可用

看到 Java 进程仍在，只能证明它尚未退出，不能证明 HTTP 请求能成功。程序可能尚在启动、依赖数据库失败或陷入死循环。后续部署同时检查进程、监听端口、日志和应用响应。

10.4 本章练习

- 不要往前翻 | 程序文件、进程和作业编号有什么区别？
- 命令填空 | 取得刚启动的后台任务 PID 使用 _____。
- 看需求写命令 | 启动一条后台 sleep，并显示其 PID 与名称。
- 看命令猜结果 | 在另一个新终端运行 jobs，会自动列出旧终端的作业吗？
- 找错误 | 把网上截图中的 PID 直接代入 kill，会有什么问题？
- 无提示实操 | 创建一条等待任务，核对身份和 PID，使用 TERM 结束，随后确认该任务不再运行。

本章新命令为 sleep / 等待、ps / process status、jobs / 作业、fg / foreground、bg / background、kill / 发信号、top / 动态进程视图。三天后解释“后台运行”为什么不等于“服务管理”。

资料依据见 S03、S12。答案见第 10 章答案。

第 11 章 systemd 与服务

本章任务 | 把已启用、正在运行和应用可用三种状态分开判断。

11.1 关掉终端以后谁照看程序

11.1.1 服务与管理器

SSH 登录服务、定时任务和应用后端往往需要在没有人打开终端时继续工作。服务是系统提供的一项持续或按需运行的功能，可以由一个或多个进程实现。systemd 是一套系统和服务管理组件，末尾 d 沿用 daemon 的命名习惯；daemon 通常指在后台提供服务的程序。

systemd 用 unit，即单元，描述管理对象。.service 是服务单元，.socket 可以描述监听套接字，.timer 可以安排定时触发。现在先掌握 service。systemctl 是控制管理器的命令，ctl 与 control 关联；journalctl 用来查询 journal 日志。

图 11-1 服务管理关系

单元文件

路径 / 身份 / 启动命令



systemd

读取定义并管理进程



应用进程 + journal

运行状态与日志证据

11.1.2 确认管理器

BASH

```
ps -p 1 -o comm=
systemctl --version
systemctl list-units --type=service --state=running
```

PID 1 是系统的第一个用户空间进程，在本书目标环境中应核对它是否为 `systemd`。容器或特殊裁剪环境可能不同。如果显示无法连接总线、未以 `systemd` 启动，应先确认是否进入了容器或其他环境，不能把命令不通归结为拼写问题。

11.2 查询与变更要分开理解

11.2.1 start 与 enable 不回答同一个问题

`start` 请求现在启动，`stop` 请求现在停止。`enable` 配置适当的启动依赖关系，通常用于以后开机自动启动，并不总是立即运行。`enable --now` 组合配置与立即启动。`disable` 通常也不自动停止当前进程。`is-active` 查询当前活动状态，`is-enabled` 查询启用状态。

本章先查询，不停止系统自带关键服务。后面的 `labapp.service` 是我们自己建立的单元，届时再练习 `start`、`stop` 和 `restart`。`restart` 会中断正在处理的请求，修改远程访问服务尤其需要保留可恢复入口。

11.2.2 读取单元和日志

BASH

```
systemctl status sshd.service --no-pager
systemctl cat sshd.service
sudo journalctl -u sshd.service -n 30 --no-pager
```

本节以 `sshd.service` 为候选，先确认本机确实存在这个单元。`status` 显示状态摘要，`cat` 查看加载的单元文件及覆盖配置。`journalctl -u` 按单元筛选，`-n 30` 取最近 30 条，`--no-pager` 直接输出而不进入分页器。普通用户可能无权查看完整系统日志，所以查询时按本机策略使用 `sudo`。日志可能包含账户与地址，分享之前先检查。

11.3 最小单元文件怎样表达一个程序

11.3.1 读懂后续部署模板

```
INI
[Unit]
Description=Linux lab Java application
After=network.target

[Service]
Type=simple
User=labapp
WorkingDirectory=/opt/labapp/current
ExecStart=/usr/lib/jvm/java-21-openjdk/bin/java -jar /opt/labapp/current/app.jar
Restart=on-failure
RestartSec=3

[Install]
WantedBy=multi-user.target
```

这份模板暂时只读，第 24 章创建用户和文件后再使用。Unit 描述单元及顺序关系，Service 描述进程，Install 说明启用时连接到哪个目标。User 指定运行账户，WorkingDirectory 给相对路径一个明确起点，ExecStart 写实际启动命令。Restart=on-failure 在异常退出时尝试重启，但不能保证应用能自动修复错误。

After 只表达顺序，不单独拉起另一个单元，也不保证网络已可访问互联网。Type=simple 下 systemd 可以较早认为启动已开始，仍要做应用层健康检查。ExecStart 默认不交给交互式 Bash 解释，不能随意塞入 >、| 或 ~ 并期待与终端相同。

11.3.2 修改单元与修改程序

修改单元文件后，需要 daemon-reload 让管理器重新读取单元定义；它不会自动重启已有程序。修改应用自己的配置，则应按应用支持的机制 reload 或 restart。reload 并非每个服务都支持。把这三类动作分清，排错会少很多歧义。

实机记录 | 本次目标机使用 systemd 255，sshd.service 查询与日志检查通过，第 24 章最终服务模板已完成启动、升级、回滚与停止验证。本文概念片段用于解释结构，完整部署以第 24 章为准。

11.4 本章练习

- 不要往前翻 | `start`、`enable`、`daemon-reload` 分别改变什么？
- 命令填空 | 查看某单元日志使用 `journalctl _____ labapp.service`。
- 看需求写命令 | 查询 `sshd.service` 的状态与最近 20 条日志。
- 看命令猜结果 | `enable` 一个服务后，它必定已经处理请求了吗？
- 找错误 | `ExecStart` 中直接写 `java ... > app.log` 为什么不能按 `Bash` 方式理解？
- 无提示实操 | 只读检查一个实际存在的服务，记录单元位置、主进程、当前状态和最近一条日志的含义。

本章新命令为 `systemctl / system control`、`journalctl / journal control`。复用 `ps` 与权限知识，七天后再解释一次服务“启用”与“可用”的区别。

资料依据见 S09、S10。答案见第 11 章答案。

第 12 章 Linux 网络基础

本章任务 | 根据地址、路由、名称和端口逐步收窄网络问题。

12.1 为什么程序说连不上

12.1.1 先把四个对象分开

网卡是网络接口，IP 地址帮助定位网络中的接口，路由决定数据下一步往哪里走，端口帮助把到达主机的流量交给对应程序。DNS 把域名解析成地址等记录。域名能解析，不代表端口开放；端口有监听，也不代表应用能正常处理请求。

IPv4 地址如 192.168.56.10，/24 是网络前缀长度，表示前 24 位属于网络前缀。这里的地址仅用于说明，实验应替换成自己虚拟机实际获得的地址。127.0.0.1 是本机回环地址，从另一台电脑访问它，连接的是那台电脑自己。

图 12-1 一次请求经过的对象

客户端

DNS 查询与目标地址



网络路径

接口 / 路由 / 允许的流量



服务器

监听端口与应用处理

12.1.2 逐层观察

BASH

```
ip -brief address
ip route
nmcli device status
nmcli connection show
```

ip 的名称关联 Internet Protocol，来自 iproute2 工具集；address 显示接口地址，route 显示路由。-brief 压缩输出。nmcli 是 NetworkManager 的命令行接口，device 显示设备，connection 显示连接配置。设备名与连接名可能不同，不要把网卡一律写成 eth0。

12.2 从本机到应用逐步检查

12.2.1 地址、路由与名称

先确认接口存在、处于可用状态并获得地址，再看默认路由。虚拟机若使用 NAT，网关通常来自虚拟化软件的虚拟网络。不要把网上的网关地址照搬到自己环境。

BASH

```
getent hosts www.openeuler.org
ping -c 3 127.0.0.1
```

getent hosts 通过系统名称服务查询地址。ping 是发送回显请求的诊断工具，名字来自回声探测的说法，不需要把它硬展开成一个正式英文缩写。-c 3 限制次数。ping 本机成功只能证明这一小段回环路径可用，ICMP 被阻止时 ping 失败也不能直接证明 Web 服务不可达。

12.2.2 检查监听端口与 HTTP

BASH

```
ss -ltn
curl -I --connect-timeout 5 --max-time 15 \
https://www.openeuler.org/
```

ss 可与 socket statistics 关联，用于查看套接字信息。套接字是程序进行网络通信使用的接口对象。-l 查看监听，-t 选择 TCP，-n 用数字显示地址端口。TCP 是提供可靠字节流传输的协议，HTTP 是常用于网页和接口请求的应用协议。

curl 是数据传输工具，名称可按项目的 client URL 关联记忆。-I 请求 HTTP 响应头，--connect-timeout 限制建立连接阶段的等待时间，--max-time 15 把整个请求限制为 15 秒。证书错误需要检查系统时间、可信证书与访问目标，不能用 -k 作为常态解决方案。离线环境无法访问外部网站时，仍可在第 22 章用本机服务练习。

12.3 NetworkManager 的配置与状态

12.3.1 一个配置可以暂时不被启用

NetworkManager 的 connection 是一组保存的网络设置，device 是实际接口。一个接口可以对应多份备用配置，通常同一时刻采用其中一份。nmcli connection show --active 显示正在使用的配置。查询得到的名称可能含空格，作为参数时需要引号。

12.3.2 配置变更应保留退路

远程改地址或断开连接会切断自己的 SSH。初次修改网络请在虚拟机控制台操作，先记录现有地址、网关、DNS 和连接 UUID。UUID 是用来唯一标识配置的一串标识符。确认虚拟网络地址规划后再使用 nmcli connection modify，应用修改也可能使当前连接断开。

本书不提供一个假定适合所有人的静态 IP 命令。地址冲突、网关错误和子网不匹配都需要具体网络信息。第 24 章保持可用的 DHCP 配置，通过 SSH 隧道访问应用即可完成部署学习。DHCP 是自动向主机提供地址等网络配置的协议。

实机记录 | 本次目标机的 NetworkManager 正在运行，ip、nmcli、DNS 查询、回环 ping 与 HTTPS 请求均完成检查。接口名和连接存储方式仍以自己的系统为准，不照搬旧网络脚本路径。

12.4 本章练习

- 不要往前翻 | IP 地址、端口、DNS 与路由分别解决什么问题？
- 命令填空 | 查看路由使用 ip _____。
- 看需求写命令 | 查看接口地址、活动连接以及 TCP 监听端口。
- 看命令猜结果 | 在电脑浏览器打开 127.0.0.1，是否自动访问虚拟机？
- 找错误 | ping 不通就断言应用停止，遗漏了哪些可能性？
- 无提示实操 | 画出宿主机、虚拟机、网关和目标服务的位置，记录实际地址，不改配置。

本章新命令为 ip / Internet Protocol 工具、nmcli / NetworkManager CLI、ping / 回显诊断、ss / socket statistics、curl / client URL。复用 getent 查询，明天从“哪一层失败”重新排查一次。

资料依据见 S11、S12、S13、S27。答案见第 12 章答案。

第 13 章 SSH 与远程服务器

本章任务 | 确认远程身份，传一份文件，再解释隧道的两端。

13.1 在自己的电脑操作虚拟机

13.1.1 客户端与服务端

SSH 是 Secure Shell，提供加密的远程登录与相关通信。你电脑上的 `ssh` 是客户端，虚拟机上的 `sshd` 是服务端守护进程。服务端需要安装、运行并监听端口，客户端要能到达这个地址。先在虚拟机控制台查询 `systemctl status sshd.service` 与 `ss -ltn`，再从电脑连接。

NAT 网络若不允许宿主机直接访问来宾，可在虚拟化软件里把宿主机 127.0.0.1 的 2222 端口转发到来宾 22。绑定回环地址可以把实验入口限制在本机。不同虚拟化软件的操作界面不同，转发目标需匹配实际来宾地址与服务端口。

BASH

```
ssh -p 2222 alex@127.0.0.1
```

这条命令在宿主机终端运行，前提是已建立上述转发且账户名为 `alex`。`-p` 指定服务端口。若采用可直接访问的仅主机网络，则改用实际虚拟机地址与相应端口，不必保留 2222。[S14]

图 13-1 本地转发的两端

电脑 127.0.0.1:18080

客户端浏览器入口



加密 SSH 连接

通过已验证的服务器



服务器 127.0.0.1:8080

应用只监听服务器本机

13.1.2 第一次连接为什么问指纹

第一次连接会显示服务器主机密钥指纹，用来确认对方身份。请在可信的虚拟机控制台查询相应主机公钥指纹，再与客户端展示的类型和值比较。`ssh-keygen` 是 SSH 密钥生成和管理工具，`-l` 显示指纹，`-f` 指定文件。例如服务器使用 `Ed25519` 主机密钥时，可由管理员读取对应公钥。

BASH

```
sudo ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub
```

只有确实存在这类密钥文件时才用该路径。不要把主机私钥传到客户端。后续若指纹变化，应先调查是否重装虚拟机、切换转发目标或存在其他风险，再决定更新 `known_hosts`，不能一律忽略提示。

13.2 用公钥登录

13.2.1 私钥留在客户端

BASH

```
mkdir -p ~/.ssh  
chmod 700 ~/.ssh  
ssh-keygen -t ed25519 -f ~/.ssh/linux-lab_ed25519
```

以上三行适用于 Linux、macOS 或 Git Bash 客户端；Windows PowerShell 用户应先建立自己的 `.ssh` 目录，再调用系统 OpenSSH 的 `ssh-keygen`，不能把 `chmod` 当成 Windows 文件权限配置。`-t` 指定密钥类型，`-f` 指定保存位置。若文件已存在，请改用新名称，不要覆盖已有密钥。设置口令可以进一步保护私钥。生成的 `.pub` 文件是公钥，可以添加到服务端用户的 `~/.ssh/authorized_keys`；没有 `.pub` 后缀的是私钥，应留在客户端。

13.2.2 安装公钥后另开窗口验证

在支持 `ssh-copy-id` 的 Linux 客户端，可用它把指定公钥安装到服务器账户。名字可以按 `copy SSH identity` 理解。Windows 未必提供它，可通过已有可信连接把公钥作为一整行追加到服务器账户的 `authorized_keys`。不要覆盖文件中原有的其他公钥。

BASH

```
ssh-copy-id -i ~/.ssh/linux-lab_ed25519.pub \  
-p 2222 alex@127.0.0.1  
ssh -i ~/.ssh/linux-lab_ed25519 \  
-p 2222 alex@127.0.0.1
```

服务端 `~/.ssh` 通常设为 700, `authorized_keys` 通常设为 600, 所有者应为该账户。具体认证还受服务端策略影响。在第二个窗口验证成功以前保留第一个会话, 尤其不要提前关闭密码认证。公钥安装本身不要求修改 `sshd_config`; 后面的端口转发还要单独检查策略。

13.3 传文件与建立隧道

13.3.1 scp 的端口是大写 P

scp 关联 secure copy, 通过 SSH 传文件。下面在宿主机执行, 把已有的 `app.jar` 上传到远程家目录。第 22 章会生成这个文件。

BASH

```
scp -P 2222 app.jar alex@127.0.0.1:app.jar
```

`-P` 是 scp 的端口选项, ssh 使用小写 `-p`, 两者容易混淆。冒号分隔主机与远程路径, 冒号后 `app.jar` 是相对远程用户家目录的路径。现代 OpenSSH 的 scp 默认使用 SFTP 传输, 遇到老系统时不要用旧协议行为推断所有细节。[S14]

13.3.2 只让应用监听服务器本机

服务器上的 Java 服务若监听 `127.0.0.1:8080`, 并且服务端允许相应转发, 可从电脑建立本地转发。

BASH

```
ssh -N -o ExitOnForwardFailure=yes \  
-L 127.0.0.1:18080:127.0.0.1:8080 \  
-p 2222 alex@127.0.0.1
```

`-N` 不执行远程命令, `-L` 配置本地转发。`ExitOnForwardFailure=yes` 让建立转发监听失败时退出, 例如本机 18080 已被占用; 它不保证每次转发到应用的连接都成功。最左的 `127.0.0.1:18080` 是电脑上的监听位置, 中间的 `127.0.0.1:8080` 从服务器一侧解释。隧道保持运行时, 在电脑访问 `http://127.0.0.1:18080/health` 即可到达服务端的 `/health`。SSH 成功不保证应用端口存在, 仍需按第 12 章检查。[S14]

13.3.3 登录成功，转发仍可能被拒绝

本次 SP4 实机的配置明确设置了 `AllowTcpForwarding no`。登录与文件传输正常，转发请求却被拒绝。因此，不能仅凭 `ssh -N` 仍在运行就判定隧道可用。由管理员检查服务端的有效配置。

BASH

```
sudo /usr/sbin/sshd -T | grep -E \  
  '^(\allowtcpforwarding|disableforwarding|permitopen) '
```

若存在 `Match` 规则，还要结合实际登录用户、客户端地址等条件，用 `sshd -T -C` 检查对应连接的结果。不要只看配置文件中某一行。确需完成本书实验时，由管理员先备份 `/etc/ssh/sshd_config`，检查已有规则，再在文件末尾为实际学习用户配置限定目标的本地转发。以下 `alex` 要换成真实账户名，不能直接套用于所有用户。[S31]

TEXT

```
Match User alex  
    AllowTcpForwarding local  
    PermitOpen 127.0.0.1:8080
```

这段是 `sshd` 配置文件内容，不是 `Shell` 命令。`Match` 后续的指令属于对应条件范围，不能在它后面随意追加全局配置。若已有更早的匹配项或其他限制，应由管理员合并核对。保存后先验证语法，成功才重新加载。

BASH

```
sudo /usr/sbin/sshd -t && sudo systemctl reload sshd
```

保留原管理会话，另建隧道并实际请求 `/health`。实验结束后，若这项授权只为测试而设，应恢复备份、再次检查语法并重新加载。本次验收只给独立测试账户临时开放该目标，验证结束已恢复原配置，没有关闭防火墙或 `SELinux`。

13.4 本章练习

- 不要往前翻 | 主机密钥和用户登录密钥分别证明谁的身份？
- 命令填空 | scp 指定端口使用 _____，ssh 使用 _____。
- 看需求写命令 | 通过 2222 端口以 alex 身份登录本机转发的虚拟机。
- 看命令猜结果 | 在远程 SSH 提示符执行 pwd，显示哪台系统的目录？
- 找错误 | 把私钥上传到服务器当作 authorized_keys 内容，错在哪里？
- 无提示实操 | 建立可信登录，上传一个普通文本文件，核对内容，再正常退出。暂不关闭任何已有认证方式。

本章新命令为 ssh / Secure Shell、ssh-keygen / SSH key generation、ssh-copy-id / 公钥安装、scp / secure copy。退出远程 Shell 可以用 exit，意为退出。本次已在 SP4 核验密钥登录、文件传输与限定目标的隧道；其他安装方式的策略仍以实际配置为准。

资料依据见 S14、S27、S29。答案见第 13 章答案。

第 14 章 压缩、打包与归档

本章任务 | 打包、校验、恢复自己的文件，并比较恢复内容。

14.1 把多个文件装进一个文件

14.1.1 归档与压缩分开看

传输一整组文件时，逐个复制容易漏掉目录结构。归档把多个文件与元数据组织成一个文件，压缩则用更紧凑的编码减少体积。tar 的名字来自 tape archive，最初与磁带归档有关，现在常用于普通文件。gzip 关联 GNU zip，一般压缩一个数据流。tar.gz 通常表示先归档再 gzip 压缩。

BASH

```
mkdir -p ~/linux-lab/archive/source
cd ~/linux-lab/archive
printf '%s\n' 'alpha' > source/a.txt
printf '%s\n' 'beta' > source/b.txt
tar -czf notes.tar.gz -C source .
tar -tzf notes.tar.gz
```

c 是 create，新建归档；z 使用 gzip；f 指定归档文件名。-C source 要求 tar 先切换到 source，再处理后面的。这样包内名称相对于 source，不会把整段家目录路径打进去。t 是 list，查看目录清单，不执行解包。

14.2 恢复要在新目录中演练

14.2.1 看过清单再解开

BASH

```
mkdir restore
tar -xzf notes.tar.gz -C restore
ls -l restore
cmp source/a.txt restore/a.txt
```

x 是 extract，提取。-C restore 指定已有的解包目录。cmp 关联 compare，逐字节比较两个文件，相同通常不输出文字并成功退出。这里比较一份样本，在真实备份中还要检查完整文件清单、权限、必要元数据以及应用能否使用恢复后的文件。

对陌生归档，应先列出内容，在隔离的新目录、普通用户身份下检查。归档中可能有意外路径、链接或覆盖目标，不能直接用 `root` 解到 `/etc` 或根目录。不要把“`tar` 有某些路径保护”理解成允许任意信任来源。

14.2.2 文件存在不等于备份可靠

把归档保存在同一块磁盘上，原盘故障时仍可能一起丢失。可用的备份至少要有清楚的内容范围、独立保存位置和恢复验证。运行中的数据库与不断变化的文件可能需要专门的一致性备份流程，仅用 `tar` 不能保证一致状态。

14.3 校验内容变化

14.3.1 为归档生成摘要

BASH

```
sha256sum notes.tar.gz > notes.tar.gz.sha256  
sha256sum -c notes.tar.gz.sha256
```

`sha256sum` 计算 SHA-256 摘要，`-c` 按校验文件验证。摘要改变说明字节内容不同，可能来自传输损坏或后续修改。若攻击者能同时替换归档与摘要，这种比较不能单独证明发布者身份，需要可信的签名或传递途径。

14.3.2 命令写反会改变任务

`tar -czf backup.tar.gz source` 创建，`tar -xzf backup.tar.gz` 提取。同样的几个字母组合，一位不同就会执行不同动作。先口头读出“我要创建”“我要列出”还是“我要恢复”，再决定 `c`、`t`、`x`，能比死背一整条命令更稳。

14.4 本章练习

- 不要往前翻 | 归档和压缩各负责什么？
- 命令填空 | 只看压缩归档清单使用 `tar -_____ notes.tar.gz`。
- 看需求写命令 | 把 `source` 内的内容归档，并恢复到新的 `restore2` 目录。
- 看命令猜结果 | `tar -C source .` 中的 `.` 指向执行命令前的目录吗？
- 找错误 | 备份只验证“文件有 1 MB”，缺少哪些检查？
- 无提示实操 | 归档自己的一个实验目录，生成摘要，在新目录恢复，比较文件内容，写出恢复步骤。

本章新命令为 `tar / tape archive`、`gzip / GNU zip`、`cmp / compare`、`sha256sum / SHA-256 checksum`。复用 `cp` 和权限知识，七天后不看原命令做一次恢复。

资料依据见 S15、S04。答案见第 14 章答案。

第 15 章 磁盘、分区与文件系统

本章任务 | 从当前路径查到挂载与存储，保持原有磁盘布局。

15.1 路径背后是怎样的存储

15.1.1 设备、分区和文件系统

磁盘或虚拟磁盘是存储设备，分区把设备划分成若干区域。文件系统在一个区域上组织文件内容与元数据。挂载把某个文件系统接到现有目录树中的一个位置，那个位置叫挂载点。逻辑卷还可以在设备与文件系统之间增加可管理的一层，本书只建立概念，不求第一次学习就操作 LVM。

图 15-1 路径怎样连接到存储

磁盘 / 分区 / 逻辑卷

提供块存储区域



文件系统

组织数据与元数据



挂载点

接到目录树中的路径

15.1.2 只读查看已经存在的布局

BASH

```
lsblk -f
findmnt /
df -h .
df -i .
```

lsblk 关联 list block devices，列出块设备；-f 显示文件系统信息。findmnt 关联 find mounts，查询挂载。df 与 disk free 关联，报告文件系统空间统计；-h 用适合阅读的单位，-i 查询 inode 使用情况。命令中的 . 让 df 找到当前目录所在文件系统。

看到 `/dev/vda`、`/dev/sda` 或 `/dev/nvme0n1` 时，不要假定哪一个一定是数据盘。它们是设备名称，随虚拟硬件和平台变化。`/` 显示的是挂载点，通常由某个分区或逻辑卷提供，不能反过来把一个路径直接称为“整块磁盘”。

15.2 明明还有空间为什么写不进去

15.2.1 块空间与 inode 都可能用完

普通文件的数据要占空间，文件对象也需要元数据资源。大量小文件可能在字节空间用完之前消耗 inode。`df -h` 与 `df -i` 应一起查看。此外，配额、只读挂载和权限问题也会拒绝写入，不一定是磁盘满。

BASH

```
du -sh ~/linux-lab
```

`du` 关联 disk usage，统计路径涉及的磁盘使用量。`-s` 只给汇总，`-h` 使用可读单位。`du` 与 `df` 的统计对象不同，二者不必完全相等。一个文件已经从目录中删除，但仍被进程打开，可能不再被 `du` 找到，却仍占文件系统空间。这与第 4 章删除名称、第 10 章进程持有文件的知识相连。

15.2.2 找到问题再回收空间

先确定哪个文件系统紧张，再定位具体目录。不要直接清空 `/var/log`，也不要对 `/tmp` 的全部内容执行不加判断的删除。日志可能正在被服务写入，临时文件也可能属于运行中的程序。真实系统应按服务日志轮转和数据保留策略处理。

15.3 挂载点会暂时遮住原内容

15.3.1 一次安全的内存文件系统实验

本节需要管理员权限，只在专用虚拟机进行。先建立新的空目录，确认它不是既有挂载点。`tmpfs` 是用内存及可能的交换空间支持的临时文件系统，不会在指定磁盘上格式化分区。下面限制最大容量为 16 MiB，实验只在本次运行期间存在。

BASH

```
mkdir -p ~/linux-lab/mount-demo
ls -la ~/linux-lab/mount-demo
findmnt --mountpoint "$HOME/linux-lab/mount-demo"
```

只有目录为空、`findmnt` 未显示已有挂载时，才执行挂载。

BASH

```
sudo mount -t tmpfs -o size=16M,mode=0755 \  
    tmpfs "$HOME/linux-lab/mount-demo"  
findmnt --mountpoint "$HOME/linux-lab/mount-demo"  
df -h "$HOME/linux-lab/mount-demo"  
sudo umount "$HOME/linux-lab/mount-demo"
```

mount 就是挂载，`-t` 指定类型，`-o` 给选项。umount 的名称是 unmount 的历史命令拼写，少一个 `n`，输入时留意。此处挂载根目录默认属于 `root`，普通用户可能只能查看，实验无需写文件。操作时留在挂载点外，避免自己的工作目录让卸载报告 `busy`。

15.3.2 不在初学实验里格式化真实盘

mkfs 可与 `make filesystem` 关联，会创建文件系统并可能破坏原有数据。fdisk 用于分区管理，dd 用于低层数据复制与转换。这些工具必须建立在明确的设备识别和备份基础上，本书不提供一条可误执行到系统盘的格式化命令。未来学习时应另加专用空虚拟盘，并通过容量、序列信息和挂载状态确认对象。

实机记录 | 本次在独立实验目录完成 16 MiB tmpfs 挂载、容量查询和卸载，并确认挂载点已释放。没有修改分区或 `/etc/fstab`，读者也无需把临时挂载写入启动配置。

15.4 本章练习

- 不要往前翻 | 设备、文件系统和挂载点如何连接起来？
- 命令填空 | 查看 inode 使用量使用 `df` _____。
- 看需求写命令 | 查看实验目录所在文件系统的空间，再统计实验目录自身占用。
- 看命令猜结果 | 把文件系统挂到一个有旧文件的目录后，原来的文件一定被删除了吗？
- 找错误 | 看到 `/dev/sdb` 就假定它是空盘并格式化，依据足够吗？
- 无提示实操 | 只读绘出自己系统的根文件系统来自哪个设备或逻辑卷，记录容量与挂载点。

本章新命令为 `lsblk` / `list block devices`、`findmnt` / `find mounts`、`df` / `disk free`、`du` / `disk usage`、`mount` / 挂载、`umount` / 卸载。`mkfs`、`fdisk`、`dd` 在本章仅识别用途。七天后从路径反查设备，保持只读。

资料依据见 S12、S16、S04。答案见第 15 章答案。

第 16 章 Bash 环境与环境变量

本章任务 | 用父、子两个 Bash 观察变量是否被传递。

16.1 同样的命令为什么在另一个终端不一样

16.1.1 变量先属于当前 Shell

```
BASH
lab_message='hello environment'
printf '%s\n' "$lab_message"
bash -c 'printf "%s\n" "${lab_message-unset}"'
```

变量为一个名称保存值。赋值时等号两边不留空格，展开时加 \$。双引号允许变量展开，并保留值里的空格。第三行启动另一个 Bash，-c 指定要执行的命令文本。

`${lab_message-unset}` 表示变量未设置时用 `unset` 这个文字。这里子 Bash 默认看不到尚未导出的 `lab_message`。

16.1.2 export 把值传给后来的子进程

```
BASH
export lab_message
bash -c 'printf "%s\n" "$lab_message"'
unset lab_message
```

`export` 是导出，把变量加入随后启动程序可继承的环境。`unset` 取消设置。环境变量是每个进程环境的一部分，子进程改自己的环境不会倒过来修改父 Shell，也不会让其他已有终端同步更新。敏感变量不适合随意打印到日志或故障截图中。

16.2 PATH 怎样找到程序

16.2.1 按目录顺序查找

```
BASH
printf '%s\n' "$PATH"
type bash
command -v bash
```

这里查询已经在使用的 Bash；Java 要到第 22 章才准备。PATH 保存一组用冒号分隔的目录。Bash 对外部命令进行路径查找时会使用它。command -v 查询命令怎样被解析，可用于脚本判断可用性。command 是 Bash 内建工具，不需要把名字编成缩写。

图 16-1 PATH 的目录次序

输入外部命令名

先完成 Shell 自身的解析



PATH 中的目录

按配置次序查找



匹配的可执行程序

路径确定后启动

如果当前目录有 report.sh，直接输入 report.sh 不一定能找到，因为当前目录通常不在 PATH 中。使用 ./report.sh 明确指出路径。不要把 . 放到 PATH 开头，那会让工作目录里同名程序优先于可信系统工具。

16.2.2 临时设置比全局修改更容易检查

BASH

```
LC_ALL=C sort ~/linux-lab/text/fruit.txt
```

这只为一次命令设置区域相关环境，便于得到稳定的字节排序。命令结束后当前 Shell 的长期设置不因此改变。相比直接修改 /etc 中的全局环境文件，这种局部设置更适合实验。

16.3 启动文件何时读取

16.3.1 交互与登录是两个维度

Bash 的启动文件取决于启动方式。交互式非登录 Bash 通常读取 ~/.bashrc；登录 Bash 读取 /etc/profile，再按规则选择用户的登录初始化文件。发行版可能通过某个文件继续加载另一个文件。非交互脚本的规则又不同，因此“我写进 .bashrc，所有服务都会得到它”并不成立。

systemd 服务通常不启动登录 Shell，不会自动继承你终端里的 `export` 和别名。服务应在自己的配置中声明必要环境，并在 `ExecStart` 使用经过验证的程序路径。

16.3.2 source 会在当前 Shell 中执行

`source` 文件路径让 Bash 在当前进程中执行文件内容；点命令 `.` 也有相关用法。只有确认文件内容可信时才执行，不能把它当成纯粹“读取文本”。一个写错的 `source` 文件可以改变当前目录、变量和函数，甚至执行其他操作。编辑初始化文件前先复制备份，再另开终端验证，保留当前可用会话。

16.4 本章练习

- 不要往前翻 | 普通变量与导出变量的差异在哪里？
- 命令填空 | 让后续子进程继承变量使用 _____ `lab_message`。
- 看需求写命令 | 设置带空格的变量并完整输出。
- 看命令猜结果 | 子 Bash 执行 `unset`，会自动移除父 Shell 的同名变量吗？
- 找错误 | `PATH="."` 为何可能使常用命令找不到，并引入执行错误程序的风险？
- 无提示实操 | 用父、子两个 Bash 证明 `export` 的作用，结束后清理自己创建的变量。

本章新命令为 `export` / 导出、`unset` / 取消设置、`command` / 命令查询与执行控制、`source` / 在当前环境执行文件。复用 `type` 和 `printf`，三天后解释服务为什么看不到终端变量。

资料依据见 S03、S09。答案见第 16 章答案。

第 17 章 管道与重定向

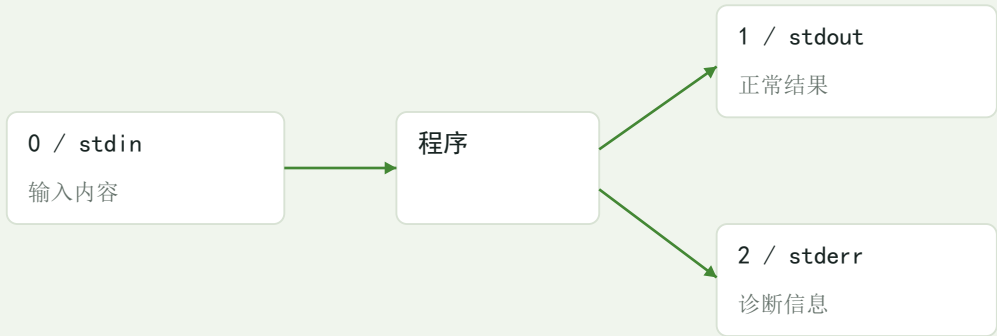
本章任务 | 预测每种输出去向，再用文件验证重定向顺序。

17.1 屏幕上的两种文字

17.1.1 为输入输出编号

程序可能打印正常结果，也可能打印错误说明。系统用文件描述符识别进程打开的输入输出对象，它是进程内的小整数编号。惯例中 0 是标准输入 `stdin`，1 是标准输出 `stdout`，2 是标准错误 `stderr`。它们常连接到终端，也可以改接文件或管道。

图 17-1 三个标准描述符



每个编号都可以连接终端、文件或其他合适的对象。

17.1.2 把正常结果保存到文件

```
BASH
mkdir -p ~/linux-lab/streams
cd ~/linux-lab/streams
printf '%s\n' first > result.txt
printf '%s\n' second >> result.txt
cat result.txt
```

符号 `>` 把标准输出指向文件，通常会创建文件或截断已有内容；`>>` 追加。两次执行后应看到 `first` 和 `second` 两行。重定向由 Shell 在启动命令前安排，因此 `sort data.txt > data.txt` 可能先截断输入。正确做法是输出到另一个文件，检查成功后再明确替换。

17.2 把错误单独保留

17.2.1 一个命令同时有成功和失败对象

```
BASH
ls result.txt missing.txt > out.txt 2> err.txt
cat out.txt
cat err.txt
```

假定 `missing.txt` 不存在, `out.txt` 保存正常列出的名称, `err.txt` 保存错误诊断。
2> 明确指定标准错误描述符。看到屏幕安静不代表命令成功, 消息可能都进入文件。
命令退出状态仍需要检查, 第 19 章会继续使用。

17.2.2 重定向从左到右处理

```
BASH
ls result.txt missing.txt > combined.txt 2>&1
```

先把 1 指到 `combined.txt`, 再把 2 复制为当前 1 的目标, 所以二者都进入同一个文件。
若交换次序, 2 先复制了原来的终端目标, 再把 1 改到文件, 错误仍可能留在终端。
2>&1 中的 & 表示复制描述符目标, 与后台运行的 & 语境不同。[S03b]

图 17-3 重定向顺序改变目标

cmd >file 2>&1

1 → file

2 → file

cmd 2>&1 >file

1 → file

2 → 终端

从左到右安排输出, 2 复制的是当时 1 的目标。

先预测 | 把同一条命令的两个重定向交换位置, 先画出 1 和 2 的去向, 再运行对比。描述符复制发生在那一刻, 后面再改 1 不会让 2 自动跟随。

17.3 让一个程序的输出成为另一个的输入

17.3.1 管道连接的是流

BASH

```
printf '%s\n' pear apple pear | sort | uniq -c
```

| 把前一个命令的标准输出连接到下一个的标准输入。第一个产生三行，第二个排序，第三个统计相邻重复。普通管道不自动包含 `stderr`。参与管道的程序可以并发读写，不能理解为先把整个文件写完再启动下一个程序。

图 17-2 排序与统计的管道



箭头连接前一程序的 `stdout` 与后一程序的 `stdin`。

普通管道不会自动把 `stderr` 合并进来。

17.3.2 保存一份同时继续传递

BASH

```
printf '%s\n' alpha beta | tee captured.txt | wc -l
```

`tee` 的名字来自 T 形分流的形象，既把输入写到文件，也继续写到标准输出。此例 `captured.txt` 保留两行，末尾 `wc -l` 统计换行数。`tee` 默认会覆盖文件，`-a` 用于追加。知道权限由哪个进程使用，就能理解 `sudo command > /etc/file` 为何可能仍被拒绝，`>` 是普通 Shell 先处理的。需要管理员写配置时，后面的章节使用 `sudo install` 或经过检查的 `sudo tee`。

17.4 多行输入与退出状态

17.4.1 here-document 让模板易读

```
BASH
cat > example.conf <<'EOF'
name=linux-lab
path=$HOME
EOF
```

<< 引入 here-document，把后续多行作为输入直到结束标记。'EOF' 的引号让正文不做变量等展开，因此文件中保留字面 \$HOME。结束标记要独占一行。此方法后面用于写配置，写之前仍要检查是否会覆盖同名文件。

17.4.2 管道最后一段成功不代表前面都成功

Bash 默认用管道最后一条命令的退出状态作为整个管道的状态。set -o pipefail 可以让前面失败更容易暴露，但它也要求理解各工具的退出规则，例如没有匹配的 grep 返回 1。脚本不应只加一个选项就停止判断错误，第 19 章给出显式判断。

17.5 本章练习

- 不要往前翻 | 0、1、2 分别是什么？
- 命令填空 | 只把错误写入 err.txt 使用 _____ err.txt。
- 看需求写命令 | 把一次 ls 的正常与错误输出一一起保存。
- 看命令猜结果 | cmd 2>&1 >file 与 cmd >file 2>&1 是否相同？
- 找错误 | cat note.txt > note.txt 为什么不能当作“原样保存”？
- 无提示实操 | 用自己创建的文件触发一条可控错误，分别保存两种输出，再组成一条排序统计管道。

本章新命令为 tee / T 形分流；set 是设置 Shell 选项的内建命令。复用 printf、ls、sort、uniq、wc、cat。明天先画出描述符去向，再写命令。

资料依据见 S03a、S03b、S04。答案见第 17 章答案。

第 18 章 grep、find 与文本搜索

本章任务 | 区分内容搜索与路径搜索，并保留含空格的文件名。

18.1 内容和名字是两种搜索

18.1.1 在日志里找一段文字

BASH

```
cd ~/linux-lab/text
grep -nF 'ERROR' app.log
grep -nF 'INFO' app.log
```

grep 的名称有历史来源，与 ed 编辑器的 g/re/p 操作有关，即按正则表达式选中行并打印。不要把它当成后来随意拼出的口号。[S17] -n 显示行号，-F 把模式当作固定字符串。本例找到包含 ERROR 的行，行号让你能够回到原文上下文。

grep 默认选择匹配行，无匹配时通常返回状态 1，出错时通常返回 2。没有输出可能是没匹配，也可能是输入有误而错误输出被重定向，判断时同时看错误与状态。

18.1.2 正则表达式从三个符号开始

正则表达式是描述文本模式的规则。^ 匹配行首，\$ 匹配行尾，. 匹配一个字符。要匹配一行以 ERROR 开始，可写 `grep '^ERROR' app.log`。要查找字面 a.b，使用 `grep -F 'a.b'` 文件最直接；普通正则中的点需要转义才表示字面点。

BASH

```
grep -nE '^(WARN|ERROR)' app.log
```

-E 使用扩展正则，圆括号分组，| 表示候选之一。这里打印以 WARN 或 ERROR 开头的行。正则模式放进单引号，避免 Bash 先解释其中符号。Shell 的 *.log 文件名通配符与 grep 的正则表达式是不同语言，不要用同一套直觉混用。

18.2 按文件条件查找

18.2.1 find 沿目录树访问

BASH

```
find ~/linux-lab -type f -name '*.log' -print
```

find 就是查找。第一个参数是起点，`-type f` 只选普通文件，`-name` 按名称匹配，`-print` 输出路径。`'*.log'` 的引号让模式交给 find；去掉引号后，Shell 可能先根据当前目录展开，造成参数数量变化。

TEXT

```
find 起点 -type f -name '*.log' -print
|      |      |      |
|      |      |      +-- 显示结果
|      |      +----- 名称条件
|      +----- 文件类型条件
+----- 搜索范围
```

18.2.2 搜索范围本身就是保护

从 `/` 开始搜索会穿过大量系统目录，普通用户常遇到权限错误，还可能很慢。知道文件属于自己的实验时，从 `~/linux-lab` 开始即可。find 也能执行删除和其他命令，但本章先只列出路径，先证明选中的对象正确，再决定后续动作。

18.3 把路径安全交给下一个命令

18.3.1 不按空格拆文件名

BASH

```
find ~/linux-lab -type f -name '*.log' \
-exec grep -nH -F 'ERROR' {} +
```

`-exec` 指定对找到的路径执行命令，`{}` 表示这些路径的位置，末尾 `+` 允许合并多个路径为一批调用。grep 的 `-H` 强制显示文件名，方便多文件查询。本例避免把路径先按空格切开，因此含空格的文件名也能正确传入。

18.3.2 换行也可能属于文件名

Linux 文件名可以包含换行，虽然人通常不应该给工作文件取这样的名字。解析 `ls` 输出或写 `for f in $(find ...)` 会破坏这些名称。复杂流程可采用 NUL 分隔的 `find -print0` 与相应工具，或者像上面直接使用 `-exec`。初学者不必马上写复杂管道，但要知道普通文本行并非普遍安全的文件名边界。

18.4 本章练习

- 不要往前翻 | `grep` 搜什么，`find` 搜什么？
- 命令填空 | 按字面搜索包含点的字符串可使用 `grep _____ 'a.b'` 文件。
- 看需求写命令 | 列出实验目录内所有 `.txt` 普通文件。
- 看命令猜结果 | `grep '^INFO'` 会匹配前面有空格再出现 `INFO` 的行吗？
- 找错误 | `find . -name *.txt` 为什么会随当前目录内容改变行为？
- 无提示实操 | 建立两份含空格文件名的日志，分别写入正常与错误行，用一个查询找出错误及文件名。

本章新命令为 `grep` / 历史 `g/re/p`、`find` / 查找。复用单引号、管道与路径范围，三天后从需求区分“找名称”与“找内容”。

资料依据见 S17、S18、S03。答案见第 18 章答案。

第 19 章 Shell Script 入门

本章任务 | 写一份有参数检查的脚本，解释失败时的退出状态。

19.1 把重复步骤写进一个文件

19.1.1 第一份脚本

Shell Script 是由 Shell 解释执行的一组命令。脚本能保留操作步骤，但同样会重复其中的错误，所以先从只读任务开始。

BASH

```
mkdir -p ~/linux-lab/scripts
cd ~/linux-lab/scripts
vim hello.sh
```

把下一段保存为 `hello.sh` 的文件内容，保存后退出编辑器。含有 `#!/usr/bin/env bash` 的代码块是完整脚本内容，不能直接粘贴到交互终端，尤其不要在自己的登录 Shell 中试执行其中的 `exit`。

BASH

```
#!/usr/bin/env bash
name=${1:-reader}
printf 'Hello, %s\n' "$name"
printf 'Current directory: %s\n' "$PWD"
```

首行叫 shebang，提示按哪个解释器启动。`/usr/bin/env bash` 通过环境中的 `PATH` 找 Bash。`env` 就是 `environment` 工具。对系统服务等强调固定环境的场景，应使用验证过的绝对解释器路径。`${1:-reader}` 使用第一个位置参数，未设置或为空时用 `reader`。

BASH

```
bash -n hello.sh
bash hello.sh Alex
chmod u+x hello.sh
./hello.sh Alex
```

`bash -n` 只做语法检查，不执行命令；它不能证明路径、权限和业务逻辑正确。用 `bash hello.sh` 不要求脚本有执行位，但必须可读。`./hello.sh` 是直接执行路径，需要相应执行权限和有效解释器。

19.2 成功与失败需要明确判断

19.2.1 退出状态

每条命令结束时提供退出状态，通常 0 表示成功，非 0 表示其他情况，具体含义依工具而定。\$? 保存最近一个前台命令或管道的退出状态，下一条命令会覆盖它。尽量直接把要判断的命令放在 if 后，减少检查错对象的机会。

BASH

```
if test -r "$1"; then
    printf 'Readable: %s\n' "$1"
else
    printf 'Cannot read: %s\n' "$1" >&2
    exit 1
fi
```

test 的普通词义是测试，-r 检查当前身份是否可读。then 后面是条件成立分支，else 是另一分支，fi 结束 if。实际脚本应先检查参数数量，避免没有参数时继续操作；下一节会补上。退出状态只有程序自己定义的一层信息，仍应配合日志。

19.2.2 参数校验写在操作之前

BASH

```
#!/usr/bin/env bash
if [ "$#" -ne 1 ]; then
    printf 'Usage: %s FILE\n' "$0" >&2
    exit 2
fi
if [ ! -f "$1" ]; then
    printf 'Not a regular file: %s\n' "$1" >&2
    exit 1
fi
wc -l -- "$1"
```

\$# 是参数个数，\$0 是脚本调用名称，\$1 是首个参数。[是 test 的一种形式，前后空格和最后的] 都是语法所需。-ne 是数值不等于，-f 检查普通文件，! 取反。-- 表示后面的参数不再作为选项解析，防止文件名以 - 开头时被误读。

19.3 循环与函数

19.3.1 保留每个参数的完整边界

```
BASH
for file in "$@"; do
    if [ -f "$file" ]; then
        wc -l -- "$file"
    fi
done
```

“\$@” 将每个位置参数保持为独立对象，含空格的名字不会被再次拆分。for 逐个取值，do 与 done 包住动作。不要用 for file in \$(ls) 遍历文件，那会把输出文本重新按空白拆开。

19.3.2 函数只组合清楚的工作

```
BASH
show_file() {
    local file=$1
    printf 'File: %s\n' "$file"
    wc -l -- "$file"
}
```

函数把重复动作封装起来，local 限制变量在当前函数的局部范围。函数参数与脚本位置参数规则类似。起步时让一个函数完成一个可描述的小任务，不要把提权、网络下载和文件删除混进一个“万能函数”。

19.4 调试与执行边界

脚本出现语法错误，先用 `bash -n`；逻辑不对时可以用 `bash -x` 跟踪实际执行，但跟踪可能显示变量展开后的秘密内容。Windows 编辑的 CRLF 行尾有时让解释器看到额外的回车，脚本应保存为 UTF-8 与 LF 换行。set -e 有复杂例外，不应代替关键步骤的显式判断；本书第 20 章保留清楚的失败分支。

19.5 本章练习

- 不要往前翻 | 直接执行脚本与 `bash script.sh` 有什么区别？
- 命令填空 | 检查语法但不执行使用 `bash _____ script.sh`。
- 看需求写命令 | 把首个参数保存到变量并完整输出。
- 看命令猜结果 | 传入 `'my notes.txt'` 时，`"$@"` 保留几个参数？
- 找错误 | `name = Alex` 为什么不能按变量赋值理解？
- 无提示实操 | 写一个只接受一个文件参数的行数脚本，测试无参数、不存在文件和含空格文件名三种情况。

本章新语法为 `if`、`for`、函数和位置参数；新命令或内建为 `test`、`[`、`exit`、`local`、`env`。复用 `chmod`、`wc`、`printf`。明天先写错误分支，再补正常分支。

资料依据见 S03、S04。答案见第 19 章答案。

第 20 章 Shell Script 实战

本章任务 | 完成一次备份恢复，验证健康检查能够识别异常。

20.1 为自己的笔记生成可恢复归档

20.1.1 先定义输入、输出与失败条件

本章备份范围固定为 `~/linux-lab/notes`，输出位于 `~/linux-lab/backups` 的独立目录。脚本不删除旧备份，不接收任意系统路径。每次成功应生成一份压缩归档及一个 SHA256 文件。如果源目录不存在、打包失败或校验失败，就返回非零状态，保留现场供检查。

BASH

```
mkdir -p ~/linux-lab/notes ~/linux-lab/scripts  
printf '%s\n' 'Linux practice' > ~/linux-lab/notes/day1.txt
```

20.1.2 把脚本保存为 backup-notes.sh

BASH

```
#!/usr/bin/env bash
set -u
umask 077
root="$HOME/linux-lab"
source_dir="$root/notes"
backup_root="$root/backups"
if [ ! -d "$source_dir" ]; then
    printf 'Missing directory: %s\n' "$source_dir" >&2
    exit 1
fi
if ! mkdir -p -- "$backup_root"; then
    exit 1
fi
run_dir=$(mktemp -d "$backup_root/run.XXXXXXXX") || exit 1
archive="$run_dir/notes.tar.gz"
if ! tar -czf "$archive" -C "$source_dir" .; then
    printf 'Archive failed; inspect %s\n' "$run_dir" >&2
    exit 1
fi
if ! (
    cd "$run_dir" || exit 1
    sha256sum notes.tar.gz > notes.tar.gz.sha256 &&
    sha256sum -c notes.tar.gz.sha256
); then
    printf 'Checksum failed: %s\n' "$run_dir" >&2
    exit 1
fi
printf 'Backup ready: %s\n' "$run_dir"
```

mktemp 关联 make temporary, -d 创建唯一目录, XXXXXXXX 是随机名称模板。独立目录避免重复运行直接覆盖同名归档。umask 077 让新建内容默认更私有。set -u 让未设置变量的展开更容易暴露,但不能替代检查变量值是否符合业务要求。

圆括号创建子 Shell,其中的 cd 不会改变脚本其余部分的工作目录。&& 只在前一步成功时继续。摘要文件写入相对文件名,便于把整份备份目录一起搬到其他位置验证。

20.2 检验脚本是否真的可靠

20.2.1 三类测试

BASH

```
cd ~/linux-lab/scripts
bash -n backup-notes.sh
bash backup-notes.sh
```

正常运行后不要只读“Backup ready”。找到输出目录，列出归档清单，在新目录恢复并比较内容。再暂时把 notes 改名，验证脚本明确失败且不报告成功，随后恢复名称。最后在 notes 中放入含空格文件名，确认归档仍保留完整名字。每次失败都要记录退出状态与诊断位置。

20.2.2 这份脚本的边界

这是个人实验目录的归档脚本。它假设目录由当前用户控制，不为多个不可信用户共享写入的环境设计，也不保证备份期间不断变化的文件具有应用一致性。未来加入数据库、自动清理和定时执行时，需要分别设计一致性、保留策略与并发控制。

20.3 为应用写一个小型健康检查

20.3.1 只检查明确的接口

第 22 章的应用提供 /health。保存下面脚本为 check-health.sh，在应用启动后运行。

BASH

```
#!/usr/bin/env bash
url='http://127.0.0.1:8080/health'
if ! body=$(curl -fsS --noproxy '*' \
  --connect-timeout 2 --max-time 5 \
  --write-out '\n{%http_code}' "$url"); then
  printf 'Request failed: %s\n' "$url" >&2
  exit 1
fi
if [ "$body" != $'ok\n200' ]; then
  printf 'Unexpected response: %s\n' "$body" >&2
  exit 1
fi
printf '%s\n' 'healthy'
```

curl 的 -f 对 HTTP 错误状态返回失败，-s 减少进度信息，-S 在静默模式下仍显示错误。--max-time 限制总耗时，避免请求无限等待。--noproxy 指定本机请求不走代

理, `--write-out` 在正文后追加状态码。`$(...)` 捕获输出并去掉末尾换行。本例正文约定为 `ok` 加一个换行, 再接输出格式中的换行和 `200`, 因此比较值写成 `$'ok\n\n200'`。这个 Bash 语法把 `\n` 转成真正换行, 同时核对状态和正文, 拒绝携带 `ok` 文字的重定向响应。HTTP 成功也不必然等于业务健康, 检查内容应与应用约定一致。

20.3.2 故障时只提供证据

检查失败后, 查看 `systemctl status`、`journalctl` 和 `ss`。不要自动无限重启服务, 这可能掩盖根因, 甚至反复中断仍可处理的请求。自动修复应在人工理解失败原因之后再设计。

20.4 本章练习

- 不要往前翻 | 为什么每次备份生成一个新目录?
- 命令填空 | 限制 `curl` 总执行时间使用 _____ 5。
- 看需求写命令 | 先检查 `backup-notes.sh` 语法, 再运行。
- 看命令猜结果 | 健康接口返回 HTTP 200、正文 `failed`, 本章脚本会成功吗?
- 找错误 | 只在 `tar` 后打印 `done`, 却不检查 `tar` 结果, 会造成什么误判?
- 无提示实操 | 完成一次成功备份与一次源目录缺失测试, 恢复源目录后, 独立验证归档可恢复。

本章新命令为 `mktemp` / `make temporary`, 新增 `!`、`&&`、`$(...)` 与子 Shell 的组合。十分钟后解释每一个 `exit 1` 对应的失败原因, 一周后实际恢复一次备份。

资料依据见 S03、S04、S13、S15。答案见第 20 章答案。完整脚本随 `examples/` 一起提供。

第 21 章 Linux 开发环境

本章任务 | 用两次本地 Git 提交记录一个小项目的修改。

21.1 让代码、产物和记录各有位置

21.1.1 从一个项目目录开始

开发目录中常混有源代码、编译结果和运行日志。如果每次都在当前目录随意生成文件，很快会分不清该上传什么。建立一个小项目，主动区分这三类内容。

BASH

```
mkdir -p ~/linux-lab/dev/src
mkdir -p ~/linux-lab/dev/build
mkdir -p ~/linux-lab/dev/logs
cd ~/linux-lab/dev
```

src 保存源代码，build 保存可重新生成的构建结果，logs 保存运行记录。这些名称是本项目约定，不是 Linux 强制规则。发布时应能从固定源码和依赖重新生成产物，不能只靠某台电脑里一份来源不明的文件。

21.1.2 构建工具与运行时

编译器把源代码转换成另一种表示，运行时负责执行相应程序。对 Java，javac 编译 .java 为字节码，java 启动 Java 虚拟机运行它们。Maven 可以管理依赖、编译、测试和打包。这些工具需要版本记录，尤其在“本机能运行，服务器不能运行”时，版本与启动参数往往是最先要比对的信息。

21.2 用 Git 记录有意义的修改

21.2.1 本地仓库不等于公开发布

Git 是分布式版本控制工具，它的名字无需展开为一个技术缩写。git init 在当前目录建立本地仓库，git add 选择本次要记录的内容，git commit 形成一次提交。整个流程可以没有网络，也不会自动发布到 GitHub。

BASH

```
git init
printf '%s\n' 'build/' 'logs/' '*.class' > .gitignore
printf '%s\n' '# Linux lab project' > README.md
git status
git add README.md .gitignore
```

.gitignore 列出通常不应纳入版本记录的生成文件模式。已被跟踪的文件不会因为后来加入忽略规则就自动移除。密码、密钥和真实业务数据也不应提交。

21.2.2 配置自己的署名

提交需要作者名字和邮箱。使用你愿意记录到本地历史中的值，在本项目配置，避免不必要地改变全局配置。

TEXT

```
git config user.name "替换为你的名字"
git config user.email "替换为你的邮箱"
git commit -m "建立实验项目"
```

此处是需要替换的模板。提交信息说明发生了什么变化。之后修改 README.md，再运行 git diff 查看尚未暂存的差异；git diff --cached 查看已经暂存的内容。检查以后再提交，避免把临时调试信息一起保存。

21.3 可复现比依赖手工记忆更可靠

21.3.1 记录环境

项目说明中记录系统版本、架构、JDK 版本、构建命令与启动命令。依赖版本固定后，升级应作为明确变更处理。把网络下载脚本直接通过管道交给 Shell，会跳过阅读与校验，本书采用先检查来源与内容、再执行的方式。

21.3.2 把构建和部署分开

构建环境需要编译器与依赖，运行环境只需要执行应用所需的组件。初学阶段可以在虚拟机中完成两者，理解流程后再在本机构建并上传 JAR。服务器部署不应临时到任意目录寻找“最新那个 jar”，应使用明确的版本目录和文件名，第 24 章将把这一点变成操作步骤。

21.4 本章练习

- 不要往前翻 | 源代码、构建产物和日志为什么分别管理？
- 命令填空 | 查看未暂存差异使用 `git _____`。
- 看需求写命令 | 仅把 `README.md` 加入下一次提交。
- 看命令猜结果 | `git init` 是否会自动在 GitHub 建立公开仓库？
- 找错误 | 把私钥加入 Git 后再写进 `.gitignore`，能否自动从历史中消失？
- 无提示实操 | 在专用项目里完成两次本地提交，第二次只修改说明文件，并查看历史与差异。

本章新命令为 `git` / 版本控制工具。首次接触的子命令有 `init`、`status`、`add`、`commit`、`diff`、`config`；查看历史使用 `git log`。复用 `mkdir`、`printf`，明天解释一次 `add` 与 `commit` 的差别。

资料依据见 S19。答案见第 21 章答案。

第 22 章 Java 与 Linux

本章任务 | 从源文件生成 JAR，验证接口后正常停止服务。

22.1 先准备完整的 JDK

22.1.1 确认运行与编译能力

JVM 是 Java Virtual Machine，执行 Java 字节码的虚拟机。JDK 是 Java Development Kit，包含开发工具及运行所需组件。只有 `java` 命令可用，不代表 `javac` 编译器也已经安装。

BASH

```
dnf search openjdk
dnf provides '*/javac'
dnf info java-21-openjdk-devel
```

本书选择 JDK 21 作为练习基线。本次已在 SP4 x86_64 官方仓库查询并安装 `java-21-openjdk-devel`。读者仍应先确认自己的候选包、来源与交易。

BASH

```
sudo dnf install java-21-openjdk-devel
```

安装成功不代表默认命令已经指向 Java 21。本次一起安装 Maven 后，依赖中同时出现 Java 8、11 和 21，而 `java`、`javac` 默认仍选中了 Java 8。先查询 JDK 21 的文件，再验证这份工具链。

BASH

```
rpm -ql java-21-openjdk-devel | grep '/bin/javac$'
ls -ld /usr/lib/jvm/java-21-openjdk
/usr/lib/jvm/java-21-openjdk/bin/java -version
/usr/lib/jvm/java-21-openjdk/bin/javac -version
```

本次验证可用的目录为 `/usr/lib/jvm/java-21-openjdk`。若本机不同，应使用实际查到并验证过的 JDK 根目录。确认两个版本均为 21 后，在当前终端选择它。

BASH

```
export JAVA_HOME=/usr/lib/jvm/java-21-openjdk
export PATH="$JAVA_HOME/bin:$PATH"
java -version
javac -version
command -v java
command -v javac
command -v jar
```

javac 关联 Java compiler, jar 关联 Java archive。java 与 javac 应属于相容的工具链, 不能只看其中一个版本。JAVA_HOME 若被构建工具使用, 应指向 JDK 根目录, 不是 bin/java 文件; 不要从其他机器复制一个绝对目录就假定正确。

提示 | 这里的 export 只影响当前终端及其后续子进程。换一个终端、重新登录或使用 sudo 后, 都要重新核对实际 Java。systemd 不自动继承这个终端的 PATH, 第 24 章会在单元中写入已验证的 Java 绝对路径。[S30]

22.2 用标准 JDK 建立第一个 HTTP 服务

22.2.1 完整程序 LabServer.java

先建立目录, 再在编辑器中保存完整源文件。

BASH

```
mkdir -p ~/linux-lab/java-app
cd ~/linux-lab/java-app
vim LabServer.java
```

程序只监听本机回环地址, 提供 /health 与 / 两个 GET 接口, 不依赖外部 Java 库。它是教学程序, 不承担公网生产流量。

JAVA

```
import com.sun.net.httpserver.HttpServer;
import java.net.InetSocketAddress;
import java.nio.charset.StandardCharsets;

public class LabServer {
    public static void main(String[] args) throws Exception {
        int port = Integer.parseInt(
            System.getProperty("lab.port", "8080"));
        String version = "v1";
        HttpServer server = HttpServer.create(
            new InetSocketAddress("127.0.0.1", port), 0);
```

JAVA / CONTINUED

```
server.createContext("/", exchange -> {
    String path = exchange.getRequestURI().getPath();
    int status = 200;
    String text;
    if (!exchange.getRequestMethod().equals("GET")) {
        status = 405;
        exchange.getResponseHeaders().set("Allow", "GET");
        text = "method not allowed\n";
    } else if (path.equals("/health")) {
        text = "ok\n";
    } else if (path.equals("/")) {
        text = "Hola Euler " + version + "\n";
    } else {
        status = 404;
        text = "not found\n";
    }
    byte[] body = text.getBytes(StandardCharsets.UTF_8);
    exchange.getResponseHeaders().set(
        "Content-Type", "text/plain; charset=utf-8");
    if (exchange.getRequestMethod().equals("HEAD")) {
        exchange.sendResponseHeaders(status, -1);
        exchange.close();
    } else {
        exchange.sendResponseHeaders(status, body.length);
        try (var output = exchange.getResponseBody()) {
            output.write(body);
        }
    }
    System.out.println(status + " " + path);
});
Runtime.getRuntime().addShutdownHook(
    new Thread(() -> server.stop(1)));
server.start();
System.out.println("Listening on 127.0.0.1:" + port);
}
}
```

HttpServer 是 JDK 的轻量 HTTP API。InetSocketAddress 指定监听地址与端口。收到请求后，程序检查方法和路径，写回状态码与正文。200 表示这次请求成功，404 表示找不到路径，405 表示不接受该方法，并通过 Allow 说明支持 GET。即使拒绝 HEAD，也不发送响应正文。字符先编码成 UTF-8 字节，再把字节长度交给响应接口，避免把中文字符数误作字节数。关闭钩子在正常终止流程中停止服务，不能承诺在强制 KILL 下仍执行。

22.2.2 编译、打包、前台运行

BASH

```
cd ~/linux-lab/java-app
mkdir -p classes
javac --release 21 -d classes LabServer.java
jar --create --file app.jar --main-class LabServer \
  -C classes .
java -jar app.jar
```

`--release 21` 指定面向 Java 21, `-d` 指定字节码输出目录。 `jar --create` 创建归档, `--file` 指定文件, `--main-class` 写入启动入口, `-C classes .` 把 `classes` 中的内容放进包。 `java -jar` 按 JAR 的入口运行。此时程序占用前台终端, 在第二个终端检查。

BASH

```
curl -fsS http://127.0.0.1:8080/health
curl -fsS http://127.0.0.1:8080/
ss -ltn
```

预期健康正文为 `ok`, 根路径正文为 `Hola Euler v1`。确认 `ss` 显示的监听地址是 `127.0.0.1`。完成后回第一个终端 `Ctrl+C`, 释放端口。若出现地址已使用, 先查占用者, 不要随意结束其他 Java 进程。

实测提示 | 本次 `ss` 将监听显示为 `[::ffff:127.0.0.1]:8080`, 这是 IPv4 映射形式的回环地址。检查时要结合完整地址, 不能只因出现冒号就断言应用监听了所有网卡。

22.3 把同一目标换成 Spring Boot

22.3.1 固定一份独立项目

Spring Boot 可以组织 Web 应用及其依赖。本书固定 3.5.16 作为另一条练习路径, 官方该版本文档要求至少 Java 17, JDK 21 在其声明的兼容范围内; Maven 至少 3.6.3。[S21] 这是刻意固定的教学版本, 后续安全维护仍要重新核对版本支持情况。

先建立独立项目及其源码、资源目录, 再创建 `pom.xml`。

BASH

```
mkdir -p ~/linux-lab/spring-lab/src/main/java/book/alex
mkdir -p ~/linux-lab/spring-lab/src/main/resources
cd ~/linux-lab/spring-lab
vim pom.xml
```

POM 是 Project Object Model，描述项目坐标、依赖与构建。下面完整配置也随 examples/spring-lab 提供。

XML

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.5.16</version>
    <relativePath/>
  </parent>
  <groupId>book.alex</groupId>
  <artifactId>spring-lab</artifactId>
  <version>1.0.0</version>
  <properties>
    <java.version>21</java.version>
  </properties>
  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
  </dependencies>
  <build>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
      </plugin>
    </plugins>
  </build>
</project>
```

22.3.2 一个控制器与两个接口

在 src/main/java/book/alex/LabApplication.java 保存以下内容。package 声明必须与目录对应。

JAVA

```
package book.alex;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
```

JAVA / CONTINUED

```
@SpringBootApplication
@RestController
public class LabApplication {
    public static void main(String[] args) {
        SpringApplication.run(LabApplication.class, args);
    }
    @GetMapping("/health")
    public String health() { return "ok\n"; }

    @GetMapping("/")
    public String home() { return "Hola Euler Spring\n"; }
}
```

在 `src/main/resources/application.properties` 写入以下两行，使默认监听仍然限制在本机。

PROPERTIES

```
server.address=127.0.0.1
server.port=8080
```

22.3.3 构建前核对 Maven

通过 `dnf search maven` 与 `dnf info maven` 确认包和版本，再按第 9 章安装。`mvn` 是 Maven 的命令名。运行 `mvn -version` 检查它实际使用的 Java。首次构建需要从配置的依赖仓库下载内容，失败时分别检查仓库、代理和证书，不能把它误判为 Linux 文件权限问题。

安装 Maven 后，在当前终端重新核对本章的 `JAVA_HOME` 和 `PATH`。下面的 `mvn -version` 应显示 Java 21；若仍为 1.8，先修正工具链选择，再构建。

BASH

```
cd ~/linux-lab/spring-lab
mvn -version
mvn package
java -jar target/spring-lab-1.0.0.jar
```

先停止上一节占用 8080 的程序，再启动此应用。对 `/health` 与 `/` 重复 `curl` 检查。这个极小项目没有随附自动化单元测试，`mvn package` 成功只说明该构建过程通过，不能声称“业务测试全部通过”。部署可以使用标准 JDK 程序或这份 Spring Boot JAR，第 24 章主线采用前者减少下载依赖。

22.3.4 仓库拒绝下载时保留证据

本次测试中，Maven Central 的两个官方入口均返回 HTTP 403。原始构建失败属于依赖访问问题，不能记成默认网络条件下构建成功。随后使用阿里云公开镜像，在独立测试配置下完成了 3.5.16 构建与接口检查。读者应按自己的网络和组织规则选择可信镜像，不要关闭证书校验。[S30]

如需复用这个可选方案，可在项目目录保存 settings-test.xml。

XML

```
<settings xmlns="http://maven.apache.org/SETTINGS/1.0.0">
  <mirrors>
    <mirror>
      <id>aliyun-central-test</id>
      <mirrorOf>central</mirrorOf>
      <url>https://maven.aliyun.com/repository/public</url>
    </mirror>
  </mirrors>
</settings>
```

在已经选定 JDK 21 的终端执行以下命令。-s 只为本次调用选择配置，不必覆盖系统或个人原有的 settings.xml。-U 要求重新检查缺失的发布依赖与更新的快照，可避免直接沿用上次失败的缓存结果。

BASH

```
cd ~/linux-lab/spring-lab
mvn -U -s settings-test.xml package
```

实机记录 | 本次 SP4 上，标准 JDK 程序和 Spring Boot 程序均完成构建、健康接口、主页、回环监听与正常停止检查。Spring Boot 构建使用上述镜像条件；测试范围和失败记录见仓库的运行验收报告。

22.4 本章练习

- 不要往前翻 | JDK、JVM、javac、java 分别负责什么？
- 命令填空 | 运行可执行 JAR 使用 java _____ app.jar。
- 看需求写命令 | 编译 LabServer.java 到 classes，再打成带入口的 app.jar。
- 看命令猜结果 | 程序只监听 127.0.0.1 时，另一台电脑直接访问虚拟机 IP 的 8080 会成功吗？
- 找错误 | 编译器比运行环境新，出现字节码版本不兼容时，应比较什么？
- 无提示实操 | 完成构建、前台运行、健康检查和正常停止，把系统、JDK 及产物摘要记录下来。

本章新命令为 java / Java 启动器、javac / Java compiler、jar / Java archive、mvn / Maven。复用 ss、curl、sha256sum，三天后从空构建目录重新生成产物。

资料依据见 S20、S21、S22、S29。答案见第 22 章答案。

第 23 章 Linux 服务器部署基础

本章任务 | 画出程序、数据、运行身份和访问入口的关系。

23.1 从“运行了”走到“可以维护”

23.1.1 部署要留下哪些东西

终端里 `java -jar` 成功只是起点。离开电脑以后，程序怎样启动，日志去哪里，谁有权修改程序，版本出错怎样恢复，都需要明确。本章先做设计，下一章逐步落地。

我们把不可随意修改的发布产物放在 `/opt/labapp/releases`，每个版本一个目录。`current` 是指向当前版本的软链接。可写数据放在 `/var/lib/labapp`，日志交给 `systemd journal`。运行账户 `labapp` 没有交互登录需求，只获取读程序和写自己数据的必要权限。

图 23-1 发布目录与可写数据



23.1.2 代码与数据需要不同所有者

发布目录由 `root` 管理，运行账户只读。即使应用发生错误，也不应轻易改写下一次启动的程序文件。数据目录交给 `labapp` 写入。示例不需要让应用写 `/opt`，也不需要把系统目录改为 `777`。

23.2 启动前确认四项条件

23.2.1 程序、路径、身份和端口

核对上传文件摘要，确认它就是刚构建的产物。用 `command -v java` 与 `java -version` 检查运行环境。用 `id labapp` 和目录权限检查服务身份。用 `ss -ltn` 检查端口是否已经被占用。任何一项不明确，都应先解决再创建自动重启策略。

路径使用绝对路径。`systemd` 的工作目录、Java 路径和 JAR 路径应可读可执行；在自己家目录可用的相对路径，换成专用账户不一定仍可用。

23.2.2 防火墙与安全策略各做什么

防火墙决定哪些网络流量被允许，SELinux 等安全机制还会限制进程可访问哪些对象。二者与传统 `rxw` 权限共同影响程序行为。只在已安装对应工具时，可用以下只读查询观察状态。

```
BASH
systemctl status firewalld.service --no-pager
sudo firewall-cmd --get-active-zones
getenforce
ls -Zd /opt /var/lib
```

`firewall-cmd` 是 `firewalld` 的命令行工具，`zone` 是应用一组网络规则的区域。`getenforce` 报告 SELinux 模式，`ls -Z` 显示安全上下文。某项工具不存在不等于所有保护都关闭，应继续确认本机实际机制。不要用关闭防火墙或把 SELinux 改成宽松模式来“修好教程”。

23.3 先限制在本机，再决定公开入口

23.3.1 SSH 隧道足够完成本书实验

Java 应用监听 `127.0.0.1:8080`，宿主机通过 SSH 隧道访问。这样本书不必在来宾防火墙中开放 8080，也不要求购买云服务器。SSH 入口仍需按照第 13 章检查服务、网络和认证。

若未来要向真实用户提供服务，应另行设计域名、TLS 证书、反向代理、访问控制与日志保留。反向代理是在前端接收请求、再转交后端应用的服务。不能把本书这个没有鉴权的教学接口直接当作完整生产系统。

23.3.2 端口开放也需要证明应用正确

从服务器本机请求 `/health`，再从隧道请求。前者成功而后者失败，重点检查 SSH 转发和客户端访问地址；两者都失败，回到进程、日志和监听。使用分层证据，比连续修改几个配置后碰运气更容易找到原因。

23.4 日志、重启和升级

23.4.1 让日志能解释一次失败

日志至少要让人知道发生时间、组件、事件和错误细节，不应写密码或令牌。`journalctl` 的 `-u` 按服务筛选，`-b` 限定本次启动，`--since` 可以选择起始时间。服务进入 `restart` 循环时，查看第一条失败信息往往比只看最后一句启动失败更有效。

23.4.2 回滚恢复的是哪个部分

保留 `v1`、部署 `v2`，再切换 `current`，必要时切回 `v1`，可以回退程序文件版本。但数据库结构或可写数据如果已被新版本修改，程序回滚不自动恢复数据兼容性。本书示例不修改持久化业务数据，因而能把重点放在服务与文件版本上。

实机记录 | 本次验证时 `firewalld` 保持运行，`SELinux` 保持 `Enforcing`。服务以 `labapp` 身份运行，发布文件不可由该账户改写，日志与健康检查通过。其他安装环境仍需核对各自策略，不能通过关闭保护来换取验收通过。

23.5 本章练习

- 不要往前翻 | 为什么程序目录由 `root` 管，而数据目录由 `labapp` 写？
- 命令填空 | 查看某服务本次开机日志可组合 `journalctl -u 服务名` _____。
- 看需求写命令 | 检查 `8080` 是否监听，并请求本机健康接口。
- 看命令猜结果 | 切回旧 `JAR` 是否必然撤销数据库中的新字段？
- 找错误 | 运行失败就 `chmod -R 777 /opt/labapp`，会掩盖什么问题？
- 无提示实操 | 为自己的实验服务写一页部署说明，包含目录、身份、端口、日志和回滚顺序。

本章新工具为 `firewall-cmd` / 防火墙控制、`getenforce` / 查询 `SELinux enforcing` 状态。复用权限、网络、`systemd` 与软链接。七天后不看图重画部署目录。

资料依据见 `S09`、`S10`、`S23`、`S24`。答案见第 23 章答案。

第 24 章 综合项目

本章任务 | 部署 v1，切到 v2，再回滚并核对实际响应。

24.1 从一台新安装的 openEuler 开始

24.1.1 项目目标与验收证据

本章把前面的知识串成一次部署。结果应是一份由 systemd 管理、以普通服务用户运行的 Java 应用。它在本机响应 /health，通过 SSH 隧道可以从电脑访问，并能完成一次版本切换与回滚。

准备专用 openEuler 24.03 LTS SP4 虚拟机，创建普通学习账户并确认 sudo 授权，记录系统版本和架构。按第 9 章准备所需工具，按第 22 章安装 JDK 21。把第 22 章源文件及构建命令带入此机，生成 app.jar。每完成一阶段，记录命令、观察结果和异常，不要一次粘贴整章。

24.1.2 先检查本章名称是否空闲

```
BASH
getent passwd labapp
getent group labapp
ls -ld /opt/labapp /var/lib/labapp
systemctl status labapp.service --no-pager
```

这些对象在新实验机上应不存在，查询可能因此返回非零或“未找到”信息。如果已有同名对象，停止创建，调查其用途，不能把它们当作可覆盖的实验材料。下面假定这些名称均未被占用。

24.2 创建身份与发布目录

24.2.1 使用无交互登录需求的服务账户

先查询 nologin 的实际路径。nologin 用于拒绝交互式登录，不影响管理员按指定身份执行服务进程。useradd -r 建系统账户，-U 同时建同名组，-M 不自动创建家目录，-d 记录其家目录字段，-s 指定登录 Shell。

```
BASH
command -v nologin
```

以下假设核验结果是 `/usr/sbin/nologin`。若不同，先替换为实际路径。

BASH

```
sudo useradd -r -U -M -d /var/lib/labapp \  
-s /usr/sbin/nologin labapp  
sudo install -d -o root -g root -m 0755 \  
/opt/labapp/releases/v1  
sudo install -d -o labapp -g labapp -m 0750 \  
/var/lib/labapp  
id labapp
```

`install` 是安装文件或目录的工具，可以在创建、复制时指定所有者、组和权限。`-d` 创建目录，`-o` 指定 owner，`-g` 指定 group，`-m` 指定模式。此处的 `root` 所有权限限制服务进程改写发布文件，而数据目录允许 `labapp` 写入。

24.2.2 上传与安装产物

若在另一台机器构建，按第 13 章用 `scp` 上传到学习账户家目录，再用 `sha256sum` 比较上传前后的摘要。若在本虚拟机构建，直接使用已生成的文件。下面假定产物位于 `~/linux-lab/java-app/app.jar`。

BASH

```
sha256sum ~/linux-lab/java-app/app.jar  
sudo install -o root -g root -m 0644 \  
~/linux-lab/java-app/app.jar /opt/labapp/releases/v1/app.jar  
sudo ln -sT releases/v1 /opt/labapp/current  
ls -l /opt/labapp/current  
sudo -u labapp test -r /opt/labapp/current/app.jar
```

最后一条由 `sudo` 以 `labapp` 身份测试可读性，成功通常不输出。它验证文件访问，尚未证明 Java 能运行。当前软链接的相对目标从 `/opt/labapp` 解析，正好到 `releases/v1`。`-T` 把目标当成一个路径项；如果 `current` 已存在，应停下来调查，不能意外在它指向的目录中再创建链接。

24.3 先用服务身份前台运行

24.3.1 把程序错误与管理器错误分开

BASH

```
command -v java  
sudo -u labapp /usr/lib/jvm/java-21-openjdk/bin/java \  
-jar /opt/labapp/current/app.jar
```

先按第 22 章验证 `/usr/lib/jvm/java-21-openjdk/bin/java` 确实是 JDK 21。`sudo` 后的默认 Java 可能不同，因此这里直接指定路径。若本机 JDK 路径不同，前台命令和后面单元都要同步采用验证过的绝对路径。前台启动后，在另一个终端运行 `curl -fsS http://127.0.0.1:8080/health`。成功后 `Ctrl+C` 停止，确认端口已释放。这样可以在引入 `systemd` 之前证明身份、程序和端口的基本组合有效。

24.3.2 写入专用单元文件

建立部署目录并进入，然后把下一段保存为 `labapp.service`。

BASH

```
mkdir -p ~/linux-lab/deploy
cd ~/linux-lab/deploy
vim labapp.service
```

INI

```
[Unit]
Description=Hola Euler teaching application
After=network.target

[Service]
Type=simple
User=labapp
Group=labapp
WorkingDirectory=/opt/labapp/current
ExecStart=/usr/lib/jvm/java-21-openjdk/bin/java -jar /opt/labapp/current/app.jar
Restart=on-failure
RestartSec=3
TimeoutStopSec=15
NoNewPrivileges=true
PrivateTmp=true
ProtectSystem=full
ProtectHome=true
UMask=0027
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

`NoNewPrivileges` 限制进程及后代通过执行程序获得新权限，`PrivateTmp` 提供私有临时目录视图，`ProtectSystem=full` 使若干系统路径只读，`ProtectHome` 限制访问家目录。它们不能替代应用安全设计，而且需要与程序真实写入需求相容。示例只读取 JAR 并输出日志，不在程序目录写文件。

24.3.3 等待应用真正就绪

Type=simple 的启动请求返回时，HTTP 接口可能还未就绪。下面的脚本最多尝试十次，同时检查响应正文和 HTTP 状态，避免把另一个版本误认成成功。准备 ~/linux-lab/scripts，把它保存为 wait-ready.sh。

BASH

```
mkdir -p ~/linux-lab/scripts
vim ~/linux-lab/scripts/wait-ready.sh
```

BASH

```
#!/usr/bin/env bash
if [ "$#" -ne 1 ]; then
    printf 'Usage: %s EXPECTED_VERSION\n' "$0" >&2
    exit 2
fi
expected="Hola Euler $1"
for attempt in {1..10}; do
    if body=$(curl -fsS --noproxy '*' \
        --connect-timeout 1 --max-time 2 \
        --write-out '\n %{http_code}' \
        http://127.0.0.1:8080/); then
        if [ "$body" = "$expected"$'\n\n200' ]; then
            printf 'Ready: %s\n' "$1"
            exit 0
        fi
    fi
    if [ "$attempt" -lt 10 ]; then sleep 1; fi
done
printf 'Not ready or wrong version: %s\n' "$1" >&2
exit 1
```

--noproxy '*' 让这个明确指向本机的请求不经过代理。--write-out 追加 HTTP 状态码，程序的正文自带一个换行，所以匹配值末尾含两个换行后接 200。\$'\n' 是 Bash 的转义字符串语法。十次尝试耗尽仍未匹配时，先检查日志与版本指向，不继续下一次切换。

24.4 加载、启动与验收

24.4.1 先验证单元语法

BASH

```
sudo install -o root -g root -m 0644 \  
~/linux-lab/deploy/labapp.service \  
/etc/systemd/system/labapp.service &&  
sudo systemd-analyze verify /etc/systemd/system/labapp.service &&  
sudo systemctl daemon-reload &&  
sudo systemctl enable --now labapp.service &&  
bash ~/linux-lab/scripts/wait-ready.sh v1  
systemctl status labapp.service --no-pager
```

systemd-analyze 是 systemd 的分析工具，verify 检查单元定义。存在与本单元相关的错误时先修正，再继续。daemon-reload 更新管理器已读取的定义；enable --now 配置后续启动并立即启动。相关写入和启动操作需要管理员权限。

24.4.2 五份证据共同构成验收

BASH

```
systemctl is-active labapp.service  
systemctl is-enabled labapp.service  
ps -C java -o user,pid,ppid,args  
ss -ltn  
curl -fsS http://127.0.0.1:8080/health  
sudo journalctl -u labapp.service -n 30 --no-pager
```

确认服务活动、启用状态符合预期，匹配的 Java 进程用户为 labapp，监听限定在 127.0.0.1:8080，健康正文为 ok，日志出现相应请求。ps -C java 可能列出多个 Java 进程，需要结合完整参数与 systemctl 主 PID 判断，不能把另一应用的进程误当成功证据。

从电脑建立第 13 章的 SSH 隧道，再访问本机 18080 端口。退出 SSH 登录后，服务应仍由 systemd 管理。是否在虚拟机重启后自动恢复，需要保存工作后自行安排一次重启验收；不要在共享服务器上为了验证而突然重启。

24.5 更新到 v2，再回到 v1

24.5.1 构建一个能辨认的新版本

把 LabServer.java 中的 String version = "v1" 改为 v2，按第 22 章重新编译与打包。先核对新 JAR 和版本记录，再安装到新的、此前不存在的版本目录。

BASH

```
test ! -e /opt/labapp/releases/v2 &&
test ! -L /opt/labapp/releases/v2 &&
sudo install -d -o root -g root -m 0755 \
/opt/labapp/releases/v2 &&
sudo install -o root -g root -m 0644 \
~/linux-lab/java-app/app.jar /opt/labapp/releases/v2/app.jar
```

确认上面的安装成功后再执行切换。

BASH

```
test -L /opt/labapp/current &&
test ! -e /opt/labapp/current.next &&
test ! -L /opt/labapp/current.next &&
sudo systemctl stop labapp.service &&
sudo ln -sT releases/v2 /opt/labapp/current.next &&
sudo mv -Tf /opt/labapp/current.next /opt/labapp/current &&
sudo systemctl start labapp.service &&
bash ~/linux-lab/scripts/wait-ready.sh v2
```

执行前确认 current 是本章创建的软链接、current.next 不存在。-e 检查可解析的对象，-L 还能发现悬空链接，因此二者都要检查。&& 让前一步失败后停止后续步骤；没有输出的检查失败也应调查，不能跳过继续执行。GNU mv 的 -T 把目标当成一个路径项而非要进入的目录，-f 允许替换本章已确认的链接。切换前先停止服务，会带来短暂中断；这是单机教学流程，没有承诺零停机。此处只更换 JAR 指向，单元定义没变，无须额外 daemon-reload。

24.5.2 回滚需要再次验证

BASH

```
test -L /opt/labapp/current &&
test ! -e /opt/labapp/current.next &&
test ! -L /opt/labapp/current.next &&
test -r /opt/labapp/releases/v1/app.jar &&
sudo systemctl stop labapp.service &&
sudo ln -sT releases/v1 /opt/labapp/current.next &&
sudo mv -Tf /opt/labapp/current.next /opt/labapp/current &&
sudo systemctl start labapp.service &&
bash ~/linux-lab/scripts/wait-ready.sh v1
```

预期等待脚本输出 Ready: v1。再访问根路径时应重新看到 v1。切换中途失败可能让服务保持停止；先检查 current 与 current.next 的实际状态，保留失败现场，再恢复一个已验证版本，不能无条件继续粘贴后续命令。再次检查日志与健康接口。v1 文件仍保存在版本目录里，回滚没有依赖重新下载旧包。若将来加入数据写入，应额外处理数据格式兼容与备份。

图 24-1 版本切换与恢复



提示 | current 指向哪里，决定下一次启动从哪里读取 JAR。已经运行的 JVM 不会因软链接改变而自动换成新版本。切换之后要重新启动，并用带版本的响应核对。

24.6 遇到失败时按证据收窄范围

24.6.1 常见故障与第一项检查

- 单元提示找不到用户，先用 id labapp 检查账户创建结果与单元拼写。

- Java 无法读取 JAR，先以 labapp 身份 `test -r`，再检查父路径权限和安全策略。
- 地址已使用，先 `ss` 查询监听，再识别拥有该端口的具体进程。
- 本机请求成功、电脑失败，先确认隧道仍运行、电脑访问端口正确。
- 服务不断重启，先 `journalctl` 查最早异常，必要时停止本服务以保留可读现场。
- 执行权限错误或 203/EXEC 一类状态，检查 ExecStart 的程序路径、可执行权限和相关拒绝日志。

24.6.2 保留实验结果

最后写一份项目记录，包含系统与 JDK 版本、产物摘要、目录权限、单元内容、启动证据和回滚结果。停止实验服务可用 `sudo systemctl disable --now labapp.service`，确认只影响自己建立的服务。无须删除账户、版本文件或整个实验目录来证明学完。

实机验收 | 本次 SP4 已完成独立服务账户、JDK 21 构建、systemd 启动、隧道访问、v2 更新、v1 回滚与故障日志恢复检查。用户选择暂不重启虚拟机，因此重启后的自动恢复仍未验证。测试结束已停止并禁用实验服务，发布文件保留供复核。作者批阅、全新安装复现和重启验收仍需分别完成。

24.7 本章练习

- 不要往前翻 | 为什么要先以 labapp 身份前台运行一次？
- 命令填空 | 读取新单元定义使用 `systemctl` _____。
- 看需求写命令 | 查看 labapp 的最近 50 条日志并检查健康接口。
- 看命令猜结果 | 仅把 `current` 指向 v2，已经启动的 JVM 会自动重新加载整个 JAR 吗？
- 找错误 | 状态显示 `active` 就停止检查，遗漏了哪些证据？
- 无提示实操 | 从快照开始完成 v1 部署、v2 更新、v1 回滚，另一位读者只根据你的记录也能重做。

本章新命令为 `install` / 安装文件与目录、`systemd-analyze` / 管理器分析工具。全书使用过的路径、权限、进程、日志和网络在这里一起接受检验。

资料依据见 S09、S10、S20、S23、S24、S28、S29。答案见第 24 章答案。

练习答案

先保留自己的命令和解释，再阅读这里。路径或用户名可以不同，只要符合题目条件并能说明结果。每章按六项练习的原顺序给出参考。无提示任务的结果需要实际操作验证，文字答案不能代替执行。

第 0 章

1. 再查询目标，例如用 `pwd` 检查位置、用 `ls` 检查新文件，并结合退出状态判断。
2. `pwd`。
3. 依次执行 `mkdir -p ~/linux-lab` 与 `cd ~/linux-lab`。
4. 不会单纯因目录已存在失败，但权限、只读文件系统或同名普通文件仍可能导致失败。
5. 方括号与尖括号是语法说明记号，必须换成实际选项和路径。
6. 以普通用户登录，`cd ~/linux-lab`，再 `pwd`；记录自己的完整输出。

第 1 章

1. 内核管理资源与系统调用，发行版组合可用系统，Bash 解释命令。
2. `-m`。
3. `cat /etc/os-release` 与 `uname -r`。
4. 不会。它查询内核发布版本。
5. `root` 是用户身份，用户态与内核态是执行权限层次，不能混为一谈。
6. 验收记录至少包含 openEuler 24.03 LTS SP4、实际架构、虚拟网络模式与快照名称，不能套用书中假定值。

第 2 章

1. 终端负责交互显示，Shell 解释命令，Bash 是一种 Shell。
2. `help`。

3. `printf '%s\n' 'Linux lab'`。
4. 两行，每个文本参数都使用一次格式。
5. 先检查命令拼写，再查询是否存在，无须立刻安装新软件。
6. `type cd`、`type pwd`、`type ls`；使用 `help`、`--help` 或 `man`，分页器按 `q` 退出。

第 3 章

1. `/` 是根目录，`/root` 通常是 `root` 的家目录，`~` 由 Shell 展开到当前用户家目录。
2. ...。
3. `cd ~/linux-lab/paths/notes`。
4. 回到 `paths`。
5. 空格把名称拆成多个参数，应使用 `cd 'reading notes'`。
6. `mkdir -p ~/linux-lab/project/src ~/linux-lab/project/docs`；进入 `src` 后 `cd ./docs`。使用 `pwd` 获取实际绝对路径，说明它从 `/` 开始定位。

第 4 章

1. 不会，`touch` 主要更新时间戳，目标不存在时才常用于创建空文件。
2. `-r` 或 `--recursive`。
3. 回到 `~/linux-lab/files`，确认 `original.txt` 和 `archive` 存在，再 `cp original.txt original.bak`，`mv original.bak archive/`。若已完成删除链接目标的扩展实验，先重新创建一份 `original.txt`。
4. 两个名字关联同一文件对象，内容变化会从另一名字观察到。
5. 不合适。先列出目录内容，按需逐项处理自己的实验文件，再用 `rmdir` 删除空目录。
6. 验收应能指出源、副本与移动后的路径。删除前 `pwd` 和 `ls` 检查，只删除选中的副本；内容相同可用后续学到的 `cmp` 检查。

第 5 章

1. `cat` 适合短文本或连接输出，`less` 适合需要滚动、搜索的阅读。

2. `-n`。
3. `head -n 2 app.log` 与 `wc -l app.log`。
4. 不会, `uniq` 只合并相邻重复行。
5. Shell 可能先把输出文件截断。应 `sort fruit.txt > sorted.txt`, 检查结果后再明确替换。
6. 原名单保持不变, 排序写入另一文件, `uniq -c` 处理排序文件; 检查各组计数之和是否为六。

第 6 章

1. 普通模式把按键解释为编辑动作, 插入模式主要把按键写成正文。
2. `:wq`; `:q!`。
3. `:set number`, 然后 `:5`。
4. 删除被撤销, 该行恢复。
5. `:wq` 是 Vim 的命令行模式命令, Bash 不按 Vim 规则解释它。
6. 用 `cat` 验证保存后的实际文件, 而不只看编辑窗口。若找不到文件, 先 `pwd`, 核对打开时使用的路径。

第 7 章

1. 主组与附加组参与身份和权限组织; 附加组便于一个用户获得多个协作范围。
2. `id`。
3. `getent passwd labreader` 与 `id labreader`。
4. 不加 `-a` 会设置给定的附加组列表, 可能移除原有成员关系; 加 `-a` 才是在原列表上追加。
5. 还需 `sudo` 策略明确授予 `wheel` 或该用户相应权限。
6. 普通学习用户只管理自己文件, 管理员执行明确系统维护, 服务用户只获取应用运行必要权限, 不需要无条件 `sudo`。

第 8 章

1. 普通文件的 `x` 允许尝试执行，目录的 `x` 允许搜索与穿过目录。
2. `u+x`。
3. `chmod 600 note.txt; chmod 700 private`。
4. 并非绝对无法删除。删除主要取决于父目录权限，还要考虑 `sticky bit` 等限制。
5. `777` 开放了过多写入与执行能力，也掩盖真正的所有者、父目录或安全策略问题。
6. 用 `ls -ld` 私有目录与 `ls -l` 文件分别检查；`700` 为 `7`、`0`、`0`，`600` 为 `6`、`0`、`0`，各自能解释对应的 `rw`。

第 9 章

1. 仓库索引描述可获取的包，包安装出文件，其中部分文件是可执行程序。
2. `-qf`。
3. `dnf search vim; dnf provides '*vim'`。
4. 不一定，`100` 通常表示发现可用更新，需按工具定义解释。
5. 引入不匹配的版本、依赖与签名来源，可能让原问题变成系统包冲突。
6. 记录实际仓库与候选包名，安装前审查交易，安装后 `type`、版本查询与 `rpm` 文件归属核对。只做本机可选工具。

第 10 章

1. 程序文件是存储内容，进程是运行实例，作业编号是当前 `Shell` 对其任务的编号。
2. `#!`。
3. `sleep 120 &, lab_pid=$!, ps -p "$lab_pid" -o pid,comm`。
4. 不会。`jobs` 属于当前 `Shell` 的作业表。
5. `PID` 属于具体系统和时间，照抄可能结束无关进程；长期保存的 `PID` 也可能被复用。
6. 先查自己的 `sleep`，再 `kill -TERM "$lab_pid"`，用 `ps` 查询确认。任务已自然结束时无需继续发信号。

第 11 章

1. start 现在启动, enable 配置启用关系, daemon-reload 重新读取单元定义。
2. -u。
3. systemctl status sshd.service --no-pager; sudo journalctl -u sshd.service -n 20 --no-pager, 前提为该单元存在。
4. 不必然。enable 不等于立即启动, 运行状态也不等于应用已就绪。
5. ExecStart 默认不通过交互式 Shell 解释, > 不会自动执行 Bash 重定向。
6. 使用实际存在的单元, 分别查询 status、cat、journalctl。看不全日志时核对权限, 不编造缺失内容。

第 12 章

1. IP 帮助定位接口, 端口区分通信端点, DNS 解析名称, 路由决定转发方向。
2. route。
3. ip -brief address; nmcli connection show --active; ss -ltn。
4. 不会, 127.0.0.1 是当前主机本身。需要已建立的本地转发才可能间接到达虚拟机。
5. ICMP 可能被过滤, 网络路径、应用端口与 HTTP 状态需要分别检查。
6. 图中标明本机与来宾实际地址、NAT 或仅主机网络, 以及应用监听位置, 不修改现有可用网络。

第 13 章

1. 主机密钥帮助客户端验证服务器; 用户密钥帮助服务器验证登录账户的持有者。
2. -P; -p。
3. ssh -p 2222 alex@127.0.0.1, 前提是本书约定的转发已建立。
4. 显示远程系统中当前 Shell 的目录。
5. authorized_keys 放授权公钥, 私钥应留在客户端并保护, 不能用上传私钥代替公钥安装。

6. 上传普通文本，远程用 `cat` 与必要的摘要验证，再 `exit`。首次连接应先核验指纹。

第 14 章

1. 归档组织多个文件及元数据，压缩减少编码体积。
2. `tzf`，完整命令为 `tar -tzf notes.tar.gz`。
3. `tar -czf notes.tar.gz -C source .; mkdir restore2; tar -xzf notes.tar.gz -C restore2`。
4. 它指 `-C` 切换后的 `source` 目录。
5. 缺少清单、完整性、实际恢复和内容可用性检查，也未证明独立存储。
6. 成功标准是归档可列出、摘要校验通过、能恢复到新目录，并能证明恢复内容符合原数据。

第 15 章

1. 文件系统组织设备或逻辑卷上的内容，挂载点把它接入目录树。
2. `-i`。
3. `df -h ~/linux-lab; du -sh ~/linux-lab`。
4. 不一定，旧内容通常只是被当前挂载遮住，卸载后重新可见。
5. 不够。应结合设备来源、容量、序列信息、分区与挂载关系，专用空盘也需确认。
6. 使用 `lsblk -f` 与 `findmnt /` 追溯，不运行分区或格式化命令。

第 16 章

1. 普通变量属于当前 Shell，`export` 的变量可传给随后启动的子进程。
2. `export`。
3. `lab_message='hello world'`，再 `printf '%s\n' "$lab_message"`。
4. 不会，子进程不能这样反向改变父环境。
5. 它用当前目录替换完整 PATH，丢失系统查找目录，还可能优先运行不可信的同名文件。

6. 先观察未导出时子 Bash 的默认值，再 export 后重试，最后 unset；不修改全局配置。

第 17 章

1. 0 是标准输入，1 是标准输出，2 是标准错误。
2. 2>。
3. ls result.txt missing.txt > combined.txt 2>&1，使用本章已有对象与故意缺失的对象。
4. 不同，重定向从左到右处理，复制的是当时描述符的目标。
5. 输出重定向先截断了作为输入的文件，原内容可能丢失。
6. 检查两种输出分别进入不同文件，管道结果符合原始数据，源文件没有被原地截断。

第 18 章

1. grep 选择匹配内容的行，find 按路径和属性遍历文件对象。
2. -F。
3. find ~/linux-lab -type f -name '*.txt' -print。
4. 不匹配，^ 要求紧接行首，前面空格会改变位置。
5. 无引号的星号先被 Shell 展开，find 得到的参数可能不再是一个模式。
6. 使用 find 的 -exec grep -nH -F 'ERROR' {} +；检查含空格文件名保持完整。

第 19 章

1. 直接执行依赖执行位与 shebang 等条件；bash script.sh 显式调用解释器，脚本本身至少需可读。
2. -n。
3. name=\$1，再 printf '%s\n' "\$name"；使用前先验证参数存在。
4. 一个参数。

5. 赋值不能在等号两侧加空格，否则 Shell 可能把 name 当成命令。
6. 无参数返回用法错误，不存在文件返回失败，含空格文件名正常计数，错误信息写到 stderr。

第 20 章

1. 避免同名输出被覆盖，并保留每次运行的独立结果与失败现场。
2. `--max-time`。
3. `bash -n backup-notes.sh`，然后 `bash backup-notes.sh`。
4. 不会，脚本还检查正文是否为 ok。
5. 即使归档失败也可能打印成功，读者会错误地依赖不存在或损坏的备份。
6. 源目录缺失时应非零退出，恢复后重新运行并在新目录解包，检查文件内容与摘要。不要只检查最后一句文字。

第 21 章

1. 分开之后能明确哪些要版本管理、哪些可重建、哪些属于运行现场。
2. `diff`。
3. `git add README.md`。
4. 不会，只建立本地仓库。
5. 不能，忽略规则不清除已记录的文件或历史。真实私钥一旦暴露，应按凭据泄露流程更换，不能只删一行历史记录。
6. `git status`、`git diff`、`git add`、`git commit` 后，用 `git log` 检查两次提交；不配置远程也能完成。

第 22 章

1. JDK 提供开发工具与运行组件，JVM 执行字节码，`javac` 编译，`java` 启动程序。
2. `-jar`。

3. `javac --release 21 -d classes LabServer.java; jar --create --file app.jar --main-class LabServer -C classes .。`
4. 不能直接连接该回环监听地址，需要明确的转发或不同绑定设计。
5. 比较编译目标版本与实际运行 JVM 版本，并检查是否执行了预期的 java 路径。
6. 构建成功、健康正文、监听地址、正常停止及产物摘要都应留有记录，不能把别的平台结果填为 openEuler 通过。

第 23 章

1. 运行账户不应随意改写发布程序，只对必要的运行数据有写权限。
2. `-b`。
3. `ss -ltn; curl -fsS http://127.0.0.1:8080/health`。
4. 不会，程序文件回退不自动恢复数据库数据或结构。
5. 会掩盖具体所有者、父路径、安全上下文或配置错误，同时扩大写入权限。
6. 说明应包含各目录用途与所有者、服务身份、监听地址、日志查询和版本切换后的验证方法。

第 24 章

1. 先证明程序、服务身份和路径组合有效，之后更容易把问题定位到管理器配置。
2. `daemon-reload`。
3. `sudo journalctl -u labapp.service -n 50 --no-pager; curl -fsS http://127.0.0.1:8080/health`。
4. 不会自动整体重载。应按计划重启并验证新进程使用的版本。
5. 还缺运行身份、监听位置、健康响应、日志以及需要时的重启恢复证据。
6. 完整验收包括零状态准备、v1 启动、隧道访问、v2 可辨识输出、回到 v1、失败诊断记录。任何未执行项保持待验证。

附录 A 核心命令速查

此表帮助回忆，不代替对应章节的前提说明。带管理员权限的命令只针对自己已创建的实验服务和文件。

需求	命令示例	章节
确认位置	<code>pwd</code>	3
回到实验目录	<code>cd ~/linux-lab</code>	3
创建多层目录	<code>mkdir -p project/src</code>	3
查看含隐藏项的长列表	<code>ls -la</code>	4
查看目录自身	<code>ls -ld private</code>	8
复制文件并确认覆盖	<code>cp -i source target</code>	4
改名或移动	<code>mv source target</code>	4
删除指定实验文件前确认	<code>rm -i file</code>	4
删除空目录	<code>rmdir empty-dir</code>	4
查看文件前后几行	<code>head -n 5 file; tail -n 5 file</code>	5
分页查看	<code>less file</code>	5
搜索字面文本	<code>grep -nF 'text' file</code>	18
查找普通文件	<code>find . -type f -name '*.txt'</code>	18
私有普通文件	<code>chmod 600 file</code>	8
查看当前身份	<code>id</code>	7
查询文件所属包	<code>rpm -qf /usr/bin/bash</code>	9

需求	命令示例	章节
搜索包供应者	<code>dnf provides '*/javac'</code>	9
查看指定进程	<code>ps -p PID -o pid,ppid,comm</code>	10
查看服务状态	<code>systemctl status labapp.service</code>	11
读取服务日志	<code>journalctl -u labapp.service -n 30</code>	11
查看接口和路由	<code>ip -brief address; ip route</code>	12
查询监听	<code>ss -ltn</code>	12
检查 HTTP	<code>curl -fsS URL</code>	12
列出归档	<code>tar -tzf notes.tar.gz</code>	14
查文件系统空间	<code>df -h .; df -i .</code>	15
统计目录空间	<code>du -sh directory</code>	15
查脚本语法	<code>bash -n script.sh</code>	19

表中 PID、URL、file、source、target、directory 都是待替换对象。中文分号表示两条独立命令，不应原样复制到 Shell。

附录 B 命令名称与英文关联

“词义关联”帮助记忆用途，不等于经过历史考据的命名全称。对于历史来源，下面只保留官方材料能支持的少数例子。其余以单词本义、工具官方名称或组合读法记忆，不补写命名者故事。

命令	关联	记忆重点
pwd	print working directory	打印当前工作目录
cd	change directory	改变目录
mkdir	make directory	创建目录
ls	list	列出名称
cp / mv / rm	copy / move / remove	复制、移动、移除
rmdir / ln	remove directory / link	删除空目录、建立链接
touch	触碰	主要更新时间戳
cat	concatenate	连接输入并输出
printf	formatted printing	按格式输出
head / tail / cut	头 / 尾 / 切取	选择所需文本
sort / uniq / wc	sort / unique / word count	排序、相邻去重、统计
chmod / chown	change mode / change owner	权限与所有者分开
ps / id	process status / identity	进程与身份查询
DNF / RPM	Dandified YUM / RPM Package Manager	软件包管理
systemctl / journalctl	system control / journal control	管理服务、读日志

命令	关联	记忆重点
nmcli	NetworkManager command-line interface	网络管理命令接口
ssh / scp	Secure Shell / secure copy	远程通信与传输
tar	tape archive	名称具有磁带归档历史
grep	ed 的 g/re/p 操作	按模式选行并打印
Vim	Vi IMproved	vi 的改进编辑器
Bash	Bourne Again SHell	与 Bourne Shell 的历史联系
df / du	disk free / disk usage	文件系统统计与路径用量
lsblk / findmnt	list block devices / find mounts	设备与挂载
mktemp / javac / jar	make temporary / Java compiler / Java archive	临时对象与 Java 工具

kill 的实际核心能力是发信号；less、top、Git、ping 保留工具名称与用途即可。不要为了凑英文缩写，把错误的名称解释再教给下一位读者。

来源见 S03、S04、S05、S07、S08、S14、S15、S17、S20。

附录 C 常见报错速查

报错文字随工具版本与语言环境变化。下表保留常见英文片段和诊断方向，不声称它们是本次目标系统的逐字实测输出。

报错片段	含义与先检查的事项
command not found	名称无法解析为命令。先拼写，再 type、PATH 与包安装情况。
No such file or directory	路径组件或所需对象不存在，也可能缺解释器。先 pwd 与逐级路径检查。
Not a directory	路径中某个需要作为目录的部分实际是文件。用 ls -ld 查每层。
Is a directory	工具期望普通文件，却得到目录。先确认对象及工具是否支持目录。
Permission denied	权限或策略拒绝。检查 id、父目录 x、文件权限与安全上下文。
Operation not permitted	请求的操作受额外规则限制。核对身份、能力、挂载或文件属性。
Directory not empty	空目录删除工具发现仍有内容。列出隐藏项，再决定逐项处理。
Read-only file system	目标文件系统只读。查询挂载与系统日志，不靠 chmod 修复。
No space left on device	检查块空间与 inode，也检查配额等限制。不要先删除未知系统文件。
Address already in use	地址端口被占用。ss 查询并识别所属进程。
Connection refused	目标拒绝 TCP 连接。检查地址端口、监听及可能的拒绝规则。
Connection timed out	在期限内未连通。检查路由、防火墙、目标可达性与超时范围。
Could not resolve host	名称解析失败。getent hosts、DNS 配置与网络状态逐项检查。

报错片段	含义与先检查的事项
UnsupportedClassVersionError	Java 字节码版本不被当前 JVM 支持。比较编译目标与实际运行版本。

复现前先缩小到自己的实验对象。记录“当前身份、路径、完整命令、错误、期望结果”，比只保存一张裁掉上文的截图更容易让别人帮你诊断。不要在求助记录中留下私钥、密码或令牌。

附录 D openEuler、Ubuntu 与 CentOS 常见差异

跨发行版经验有帮助，但每次迁移都要重新确认版本和默认配置。下表比较常见工具家族，不承诺所有历史版本与裁剪环境都相同。

项目	openEuler 本书环境	Ubuntu 常见环境	CentOS 相关环境
软件包家族	RPM，常用 DNF	DEB，常用 APT/dpkg	RPM；DNF 或历史 YUM，随版本变化
包供应者	用 DNF 查询	使用该发行版工具查询	不沿用为 openEuler 的保证
网络管理	先确认 NetworkManager	可能涉及 Netplan 与不同后端	不根据旧 network-scripts 教程推断
服务名	从本机单元查询	同一软件可能名称不同	同样以实际安装为准
安全机制	核对 SELinux 和防火墙实际状态	常见 AppArmor 等机制	默认及策略随版本不同
软件源	固定目标版本与架构	使用匹配 Ubuntu 版本的源	不混入 openEuler

不要把 `sudo apt install` 后面换成一个 RPM 包名就称为迁移。除了安装工具，还要核对包拆分、配置路径、服务名称与安全策略。通用的路径、进程和 Bash 概念可继续使用，默认值需要重新查询。

本附录不提供生命周期日期表，避免把会变化的维护信息印成固定事实。选择系统时应读取相应项目当前官方生命周期资料。来源见 S01、S06、S25。

附录 E Linux 后续学习路线

E.1 把本书项目再做两次

第一次从自己的笔记重做，记录还需要翻书的步骤。第二次从干净快照开始，只拿一页部署说明完成。能解释一次失败如何定位，比记住更多孤立选项更能检验基础是否扎实。

完成标准是能找到目录、辨认权限、查询包来源、定位进程、读取日志并验证端口和应用。遇到问题时，知道下一条查询在验证哪个猜想，才适合继续扩展系统。

E.2 选择一个真实方向

准备学习 Java 后端，可以继续做数据库连接、配置分环境、测试与持续集成。每加入一个依赖，都记录启动条件、账户权限、失败表现和恢复方法。不要急着把所有组件搬到公网。

偏向系统管理，可以继续学习文件系统备份、逻辑卷、定时任务、资源限制和性能观察。磁盘写入实验另建专用虚拟盘，练习前确认没有重要数据。

偏向容器与云环境，可以在理解进程、网络 and 权限之后，再学习命名空间、cgroup、镜像与容器运行时。容器仍依赖宿主机内核，不能用“隔离”两个字省略权限与数据边界。

E.3 读一份手册并提出一个问题

选一条最近使用的命令，读本机 man 页的名称、用法、选项和退出状态，做一个足够小的实验。先写预测，再运行，再解释差异。查阅网上资料时记录版本与日期，能把一个不准确的例子纠正清楚，也是一项实际成果。

需要向社区求助时，提供可复现步骤、环境版本和脱敏后的完整报错。问题解决后补充原因与处理结果。这个记录会成为你下一次独立处理问题的起点。

参考资料与版本核验

所有条目查询日期为 2026-09-23。优先使用官方项目材料。上游当前手册只用于确认基础语义，不能证明 openEuler 目标包包含所有新功能。书中实验、图示和叙述为本项目原创设计，未复制其他教材章节。

S01 openEuler 官方下载与 SP4 镜像目录

对应章节 1、9、附录 D。适用范围为 24.03 LTS SP4。用途为 确认目标版本与架构；不是安装实测。

<https://www.openeuler.org/zh/download/>

<https://repo.openeuler.org/openEuler-24.03-LTS-SP4/ISO/>

S02 openEuler SP4 快速入门与系统信息

对应章节 1。适用范围为 24.03 LTS SP4。用途为 安装流程和版本查询；虚拟机配置为本书建议。

https://docs.openeuler.org/en/docs/24.03_LTS_SP4/server/quickstart/quick_start_server.html

https://docs.openeuler.org/en/docs/24.03_LTS_SP4/server/administration/administrator/viewing_system_information.html

S03 GNU Bash Reference Manual

对应章节 0、2、3、16 至 20。适用范围为 上游手册；SP4 自带版本待核对。用途为 Shell 语义、引号、变量、条件和参数。

<https://www.gnu.org/s/bash/manual/bash.html>

S03a Bash Pipelines

对应章节 17。适用范围为 上游稳定语义。用途为 管道连接与执行状态。

https://www.gnu.org/s/bash/manual/html_node/Pipelines.html

S03b Bash Redirections

对应章节 17。适用范围为 上游稳定语义。用途为 描述符复制、打开文件与重定向次序。

https://www.gnu.org/s/bash/manual/html_node/Redirections.html

S04 GNU Coreutils Manual

对应章节 0 至 5、8、14、17、19。适用范围为 上游 9.11 文档；不等同 SP4 包版本。用途为 通用工具语义与名称核对。

<https://www.gnu.org/software/coreutils/manual/coreutils.html>

S04a GNU touch invocation

对应章节 4。适用范围为 上游稳定语义。用途为 更新时间戳、按需创建文件。

https://www.gnu.org/s/coreutils/manual/html_node/touch-invocation.html

S04b GNU cp invocation

对应章节 4。适用范围为 上游稳定语义。用途为 递归、目标目录与复制行为。

https://www.gnu.org/s/coreutils/manual/html_node/cp-invocation.html

S04c GNU chmod invocation

对应章节 8。适用范围为 上游稳定语义。用途为 权限与符号链接处理。

https://www.gnu.org/software/coreutils/manual/html_node/chmod-invocation.html

S05 Vim User Manual

对应章节 6。适用范围为 Vim 9.2 在线帮助；基础键位通用。用途为 模式、保存和基础操作。

https://vimhelp.org/usr_02.txt.html

https://vimhelp.org/usr_toc.txt.html

S06 openEuler 常用技能与账户工具手册

对应章节 7、9。适用范围为 SP4 常用技能；上游账户语义。用途为 用户、组与系统管理；已核对目标机 wheel 策略和未授权拒绝；带密码的 sudo 交互未执行。

https://docs.openeuler.org/zh/docs/24.03_LTS_SP4/server/maintenance/common_skills/common_configurations.html

<https://man7.org/linux/man-pages/man8/useradd.8.html>

<https://man7.org/linux/man-pages/man1/su.1.html>

S07 DNF Command Reference

对应章节 9。适用范围为 DNF 4 系列文档；目标包版本待核对。用途为 查询、安装交易与 check-update 状态。

https://dnf.readthedocs.io/en/latest/command_ref.html

S08 RPM 官方查询手册

对应章节 9。适用范围为 RPM 6.0 在线文档；仅参考稳定查询选项。用途为 -q、-ql、-qf；不采用未核实新选项。

<https://rpm.org/docs/6.0.x/man/rpm.8>

S09 systemd 官方服务与执行环境手册源码

对应章节 11、16、23、24。适用范围为 上游 main，目标版本待核对。用途为 Type、ExecStart、运行身份和限制选项。

<https://raw.githubusercontent.com/systemd/systemd/main/man/systemd.service.xml>

<https://raw.githubusercontent.com/systemd/systemd/main/man/systemd.exec.xml>

S10 systemctl 与 journalctl 官方手册源码

对应章节 11、23、24。适用范围为 上游 main，目标版本待核对。用途为 启用、启动、重载与日志筛选。

<https://raw.githubusercontent.com/systemd/systemd/main/man/systemctl.xml>

<https://raw.githubusercontent.com/systemd/systemd/main/man/journalctl.xml>

S11 openEuler 网络配置与 NetworkManager 手册

对应章节 12。适用范围为 SP4 文档与上游 NetworkManager。用途为 设备、连接与 nmcli 语义。

https://docs.openeuler.org/en/docs/24.03_LTS_SP4/server/network/network_configuration.html

<https://networkmanager.dev/docs/api/latest/nmcli.html>

S12 Linux man-pages 与内核凭据说明

对应章节 1、4、7、8、10、12、15。适用范围为 通用 Linux；具体实现查本机手册。用途为 身份、进程、路径、权限与文件对象。

<https://man7.org/linux/man-pages/man7/credentials.7.html>

<https://cdn.kernel.org/doc/html/latest/security/credentials.html>

S13 curl 官方命令文档

对应章节 12、20。适用范围为 上游文档；仅使用成熟选项。用途为 HTTP、失败状态与超时。

<https://curl.se/docs/manpage.html>

S14 OpenSSH 官方手册

对应章节 13。适用范围为 OpenBSD 当前手册；目标机已确认 OpenSSH 9.6p1，登录、传输和限定目标的转发已验证。用途为 主机验证、密钥、scp 与端口转发。

<https://man.openbsd.org/ssh>

<https://man.openbsd.org/scp>

<https://man.openbsd.org/ssh-keygen>

S15 GNU tar Manual

对应章节 14、20。适用范围为 上游 1.35 文档。用途为 创建、清单、提取和归档路径。

<https://www.gnu.org/software/tar/manual/tar.html>

S16 util-linux 工具手册

对应章节 15。适用范围为 上游；目标版本待核对。用途为 lsblk、挂载与卸载。

<https://man7.org/linux/man-pages/man8/lsblk.8.html>

<https://man7.org/linux/man-pages/man8/mount.8.html>

S17 GNU Grep Manual

对应章节 18、附录 B。适用范围为 上游 3.12 文档。用途为 g/re/p 名称来源、固定字符串与正则。

<https://www.gnu.org/s/grep/manual/grep.html>

S18 GNU Findutils Manual

对应章节 18。适用范围为 上游；索引核验。用途为 find 名称匹配与 -exec；离线全文取回未成功。

https://www.gnu.org/software/findutils/manual/html_mono/find.html

S19 Git 官方文档

对应章节 21。适用范围为 当前手册的基础子命令。用途为 本地仓库与提交。

<https://git-scm.com/docs/git-init>

<https://git-scm.com/docs/git-add>

<https://git-scm.com/docs/git-commit>

S20 JDK 21 工具与 HTTP API

对应章节 22、24。适用范围为 JDK 21。用途为 jar 入口、编译目标与 HttpServer API。

<https://docs.oracle.com/en/java/javase/21/docs/specs/man/jar.html>

<https://docs.oracle.com/en/java/javase/21/docs/specs/man/javac.html>

<https://docs.oracle.com/en/java/javase/21/docs/api/jdk.httpserver/com/sun/net/httpserver/HttpServer.html>

S21 Spring Boot 3.5 系统要求

对应章节 22。适用范围为 页面版本 3.5.16。用途为 Java 与 Maven 支持范围。

<https://docs.spring.io/spring-boot/3.5/system-requirements.html>

S22 Spring Boot 首个应用教程

对应章节 22。适用范围为 页面版本 3.5.16。用途为 Maven 依赖与可执行归档插件配置。

<https://docs.spring.io/spring-boot/3.5/tutorial/first-application/index.html>

S23 firewalld 官方 firewall-cmd 手册

对应章节 23。适用范围为 上游；已查询目标 firewalld 配置并确认运行状态。用途为 活动 zone 查询；不推断默认开放规则。

<https://firewalld.org/documentation/man-pages/firewall-cmd.html>

S24 SELinuxProject getenforce 手册源码

对应章节 23。适用范围为 上游；目标策略待核对。用途为 只读模式查询；不代表服务策略已通过。

<https://raw.githubusercontent.com/SELinuxProject/selinux/main/libselinux/man/man8/getenforce.8>

S25 Ubuntu 与 CentOS 官方项目资料

对应章节 附录 D。适用范围为 查询当日页面。用途为 跨发行版工具差异；不转抄安装命令。

<https://ubuntu.com/server/docs/how-to/software/package-management/>

<https://www.centos.org/centos-stream/>

检索限制与交叉检查

部分 GNU 页面初次直连超时，后续通过官方索引与 /s/ 路径检索。freedesktop HTML 手册取回被拒绝后改读 systemd 官方仓库手册源码。openEuler 的个别猜测路径无法取回，未把它们列成已核对正文。SP4 快速入门旧子页面含其他版本残留，本书使用新版入口并与镜像目录交叉确认，未抄写其过时版本指引。

SP4 镜像目录与文档已确认存在；2026-09-24 已完成目标虚拟机运行测试，包含包安装、普通用户权限、脚本、Java、服务部署与回滚。重启恢复、安装器重装和密码交互未验收，具体结果见 review/RUNTIME_REPORT.md 与 review/VALIDATION.md。

S26 Hello 算法的阅读与练习组织

对应章节 0 至 24。用于借鉴图解、短代码实践、提示框和练习的组织方法，不作为 Linux 命令语义依据。正文与本书新增图示独立创作，没有复制其教材段落或插图。

Hello 算法如何使用本书

S27 openEuler 中文运维文档交叉核对

对应章节 1、9、12、13、23。核对安装前提、RPM 查询、DNF 仓库查询和 SSH 基本流程。SP4 常用技能页中仍有较早版本的示例包名，使用时只核对通用语义，不把样例包版本写成本机结果。网络连接的保存格式以目标系统实际查询为准。

SP4 服务器快速入门

SP4 常用技能

SP4 网络配置

24.03 LTS 的 DNF 管理说明

最后一项属于 24.03 LTS 文档，提供配置位置与工具概念参考，不用其中其他发行版本的源地址替换 SP4 软件源。

S28 GNU 目标目录语义

对应章节 4、24。核对 `ln` 与 `mv` 的 `-T`，把软链接路径当作目标条目，避免进入它指向的目录。只使用长期支持的选项，未引入当前上游新增的交换接口。

GNU 目标目录选项

GNU `ln` 说明

S29 SSH 转发失败与 HTTP 响应

对应章节 13、20、22、24。核对转发监听建立失败时的退出行为，以及 405 的 `Allow` 字段和 `HEAD` 无正文约定。

OpenSSH 客户端配置

HTTP 语义 RFC 9110

S30 Maven 工具链选择与独立镜像配置

对应章节 22。`JAVA_HOME` 与 `PATH` 的选择依据 Apache Maven 官方安装说明；镜像匹配与设置文件依据官方镜像指南。阿里云页面只用于说明本次测试所采用镜像的提供者与地址，不作为 Linux 命令语义来源。访问日期为 2026-09-24。

Maven 安装与 JDK 前提

Maven 镜像设置

阿里云 Maven 镜像说明

S31 SSH 服务端转发策略

对应章节 13、24。核对 `AllowTcpForwarding`、`Match` 与 `PermitOpen` 的语义。上游默认值不能代替发行版配置；本次 SP4 实机有效值为禁用转发，限定测试用户后才完成隧道验收。

OpenSSH `sshd_config`